PRACTICAL 1: IMPORTING DATA INTO POWER BI

1. Aim

To import structured data into Microsoft Power BI Desktop from external sources, verify the imported data, and prepare it for analysis and visualization.

2. Objectives

1. Obtain a sample dataset from Kaggle in CSV format.
2. Import the CSV file into Power BI using Get Data → Text/CSV.
3. Preview the data in the Navigator window and load it into the Power BI model.
4. Understand the Transform Data option for cleaning and shaping data before loading.
5. Import data through an OData feed and select required tables.
6. Verify the imported tables and fields in Power BI.

3. Software / Requirements

- Microsoft Power BI Desktop
- Internet connection for downloading a dataset or accessing an OData feed
- Kaggle account (optional, if the dataset requires sign-in)
- Sample Superstore / Northwind data

4. Procedure — Obtain the Dataset from Kaggle

Step 1: Open Kaggle and search for a suitable Superstore dataset.

Step 2: Filter the search results for datasets containing CSV files. For this practical, use a Super store - style sales dataset.

Step 3: Open the selected dataset and download the CSV file to the computer.

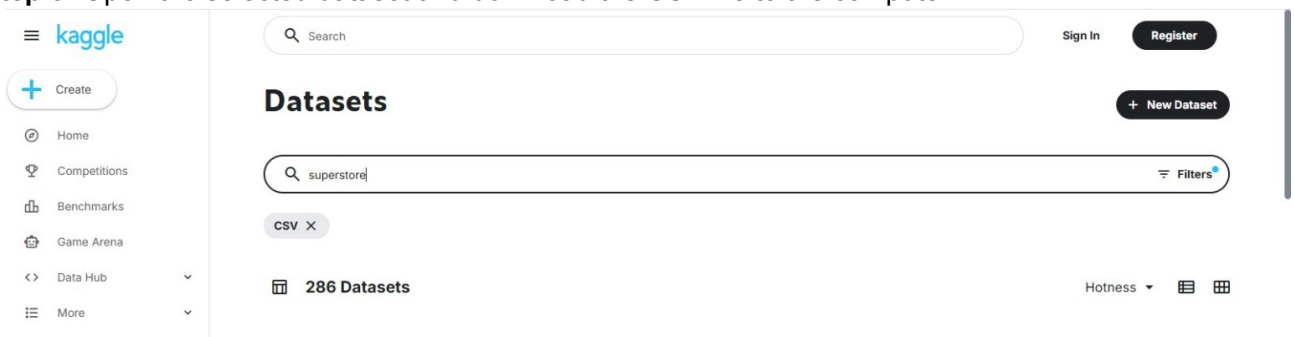


Figure 1: Searching for a Superstore dataset on Kaggle.

Step 4: Keep the downloaded CSV file in an easily accessible folder such as Downloads.

5. Import the CSV File into Power BI

Step 5: Launch Microsoft Power BI Desktop and open a blank report.

Step 6: On the Home tab, select **Get data** and choose **Text/CSV** (or use the Text/CSV connector from the data-source menu).

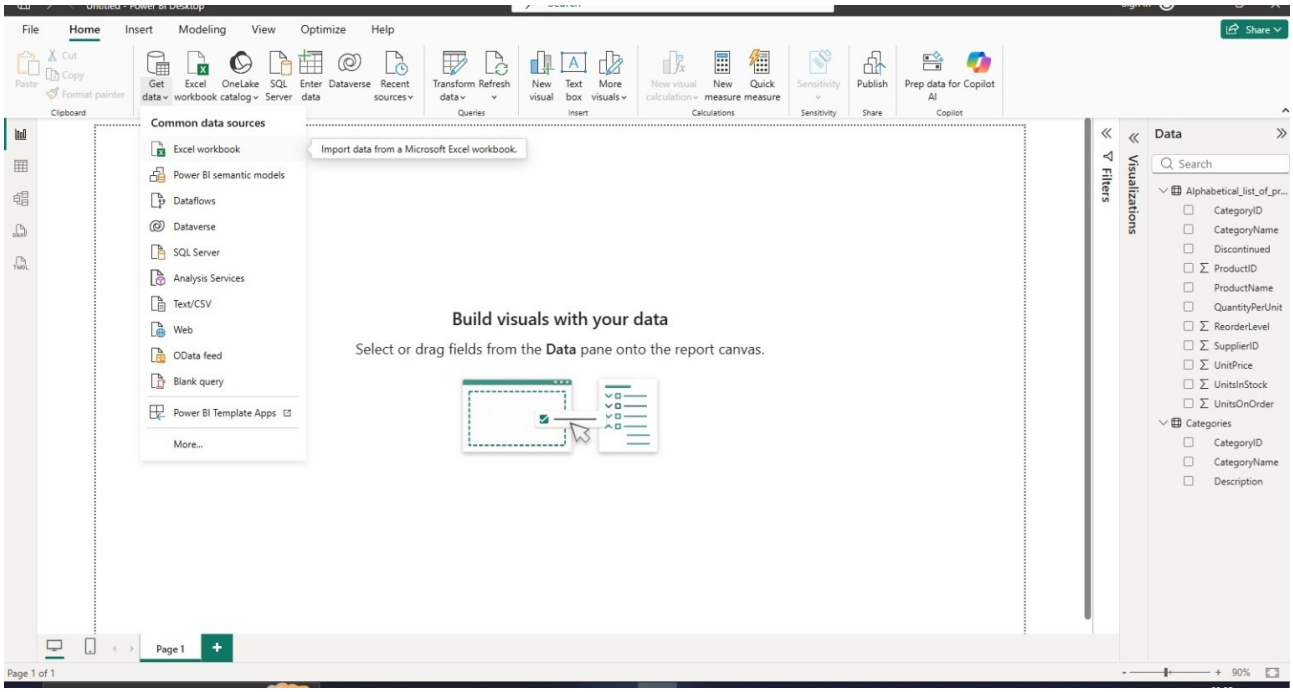


Figure 2: Power BI Desktop showing the Get Data menu and available connectors.

Step 7: When the file picker appears, browse to the downloaded CSV file and select it.

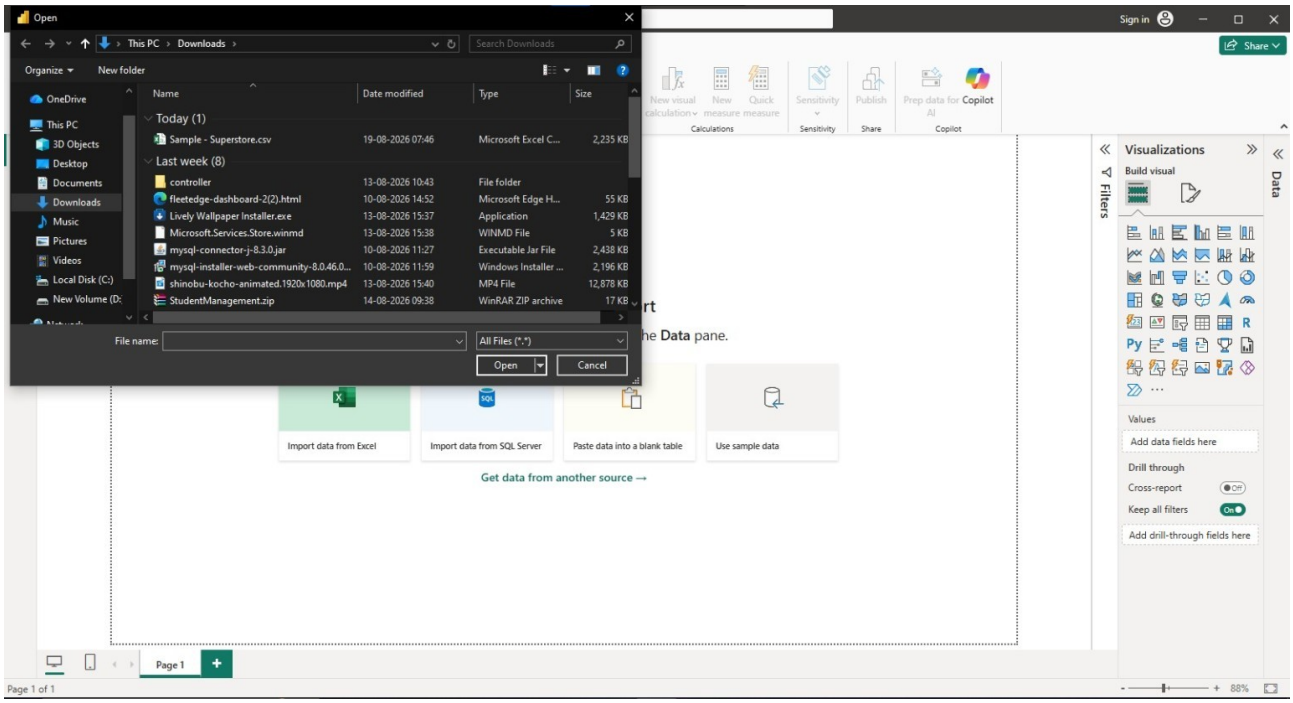
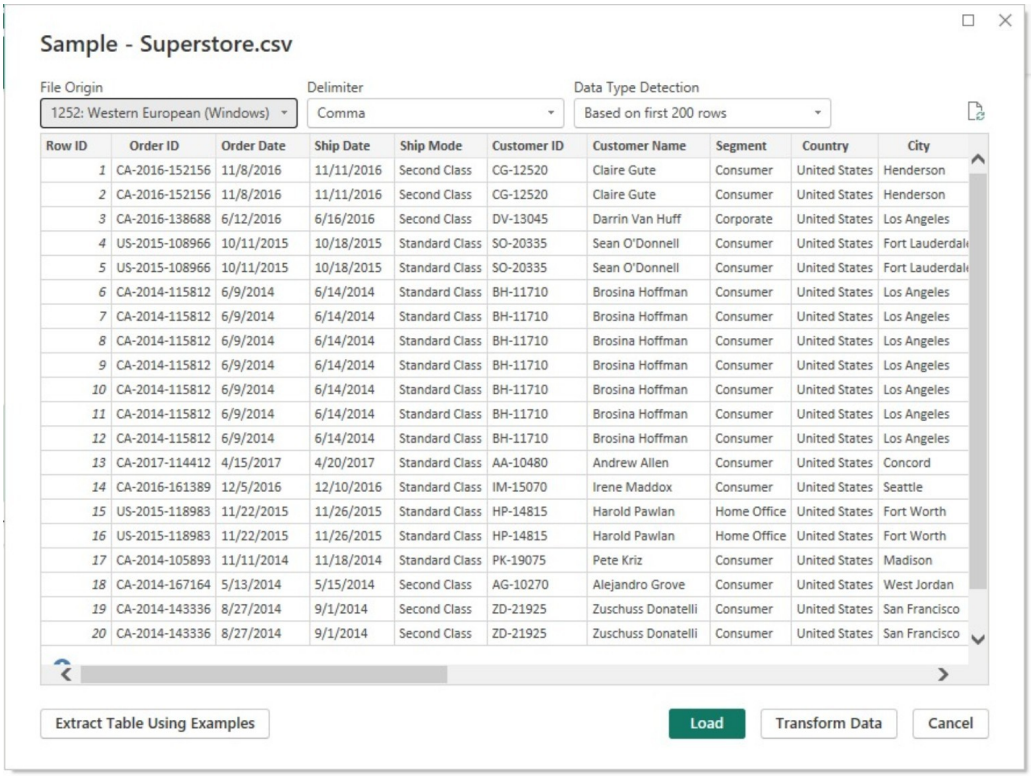


Figure 3: Selecting the downloaded Sample - Superstore.csv file.

6. Preview and Load the CSV Data

Step 8: Power BI opens a preview window. Check the file origin, delimiter, column headers, dates, and numeric fields.

Step 9: Confirm that the data is separated correctly by commas and that the first row is being interpreted as column headers.



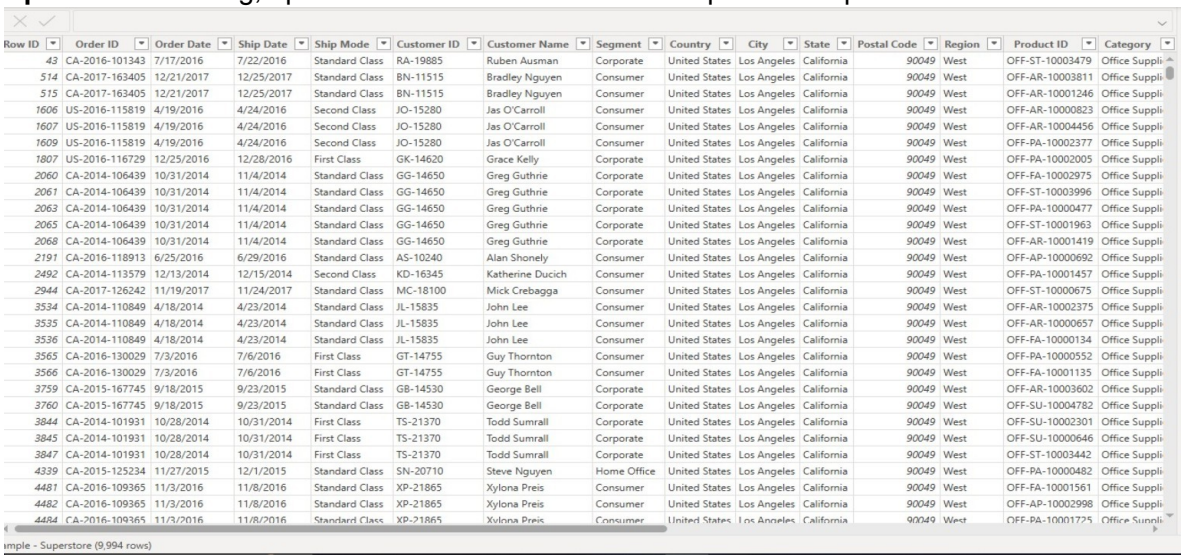
Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City
1	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson
2	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson
3	CA-2016-138688	6/12/2016	6/16/2016	Second Class	DV-13045	Darrin Van Huff	Corporate	United States	Los Angeles
4	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdal
5	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdal
6	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles
7	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles
8	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles
9	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles
10	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles
11	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles
12	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles
13	CA-2017-114412	4/15/2017	4/20/2017	Standard Class	AA-10480	Andrew Allen	Consumer	United States	Concord
14	CA-2016-161389	12/5/2016	12/10/2016	Standard Class	IM-15070	Irene Maddox	Consumer	United States	Seattle
15	US-2015-118983	11/22/2015	11/26/2015	Standard Class	HP-14815	Harold Pawlan	Home Office	United States	Fort Worth
16	US-2015-118983	11/22/2015	11/26/2015	Standard Class	HP-14815	Harold Pawlan	Home Office	United States	Fort Worth
17	CA-2014-105893	11/11/2014	11/18/2014	Standard Class	PK-19075	Pete Kriz	Consumer	United States	Madison
18	CA-2014-167164	5/13/2014	5/15/2014	Second Class	AG-10270	Alejandro Grove	Consumer	United States	West Jordan
19	CA-2014-143336	8/27/2014	9/1/2014	Second Class	ZD-21925	Zuschuss Donatelli	Consumer	United States	San Francisco
20	CA-2014-143336	8/27/2014	9/1/2014	Second Class	ZD-21925	Zuschuss Donatelli	Consumer	United States	San Francisco

Figure 4: CSV preview in Power BI before loading the dataset.

Step 10: Choose **Load** to import the data directly, or choose **Transform Data** if cleaning or reshaping is required.

7. Verify the Imported Data

Step 11: After loading, open **Table view** in Power BI to inspect the imported rows and columns.



Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	State	Postal Code	Region	Product ID	Category
43	CA-2016-101343	7/17/2016	7/22/2016	Standard Class	RA-19885	Ruben Ausman	Corporate	United States	Los Angeles	California	90049	West	OFF-ST-10003479	Office Suppli
514	CA-2017-163405	12/21/2017	12/25/2017	Standard Class	BN-11515	Bradley Nguyen	Consumer	United States	Los Angeles	California	90049	West	OFF-AR-10003811	Office Suppli
515	CA-2017-163405	12/21/2017	12/25/2017	Standard Class	BN-11515	Bradley Nguyen	Consumer	United States	Los Angeles	California	90049	West	OFF-AR-10001246	Office Suppli
1606	US-2016-115819	4/19/2016	4/24/2016	Second Class	JO-15280	Jas O'Carroll	Consumer	United States	Los Angeles	California	90049	West	OFF-AR-10000823	Office Suppli
1607	US-2016-115819	4/19/2016	4/24/2016	Second Class	JO-15280	Jas O'Carroll	Consumer	United States	Los Angeles	California	90049	West	OFF-AR-10004456	Office Suppli
1609	US-2016-115819	4/19/2016	4/24/2016	Second Class	JO-15280	Jas O'Carroll	Consumer	United States	Los Angeles	California	90049	West	OFF-PA-10002377	Office Suppli
1807	US-2016-116729	12/25/2016	12/28/2016	First Class	GK-14620	Grace Kelly	Corporate	United States	Los Angeles	California	90049	West	OFF-PA-10002005	Office Suppli
2060	CA-2014-106439	10/31/2014	11/4/2014	Standard Class	GG-14650	Greg Guthrie	Corporate	United States	Los Angeles	California	90049	West	OFF-FA-10002975	Office Suppli
2061	CA-2014-106439	10/31/2014	11/4/2014	Standard Class	GG-14650	Greg Guthrie	Corporate	United States	Los Angeles	California	90049	West	OFF-ST-10003996	Office Suppli
2063	CA-2014-106439	10/31/2014	11/4/2014	Standard Class	GG-14650	Greg Guthrie	Corporate	United States	Los Angeles	California	90049	West	OFF-PA-10000477	Office Suppli
2065	CA-2014-106439	10/31/2014	11/4/2014	Standard Class	GG-14650	Greg Guthrie	Corporate	United States	Los Angeles	California	90049	West	OFF-ST-10001963	Office Suppli
2068	CA-2014-106439	10/31/2014	11/4/2014	Standard Class	GG-14650	Greg Guthrie	Corporate	United States	Los Angeles	California	90049	West	OFF-AR-10001419	Office Suppli
2191	CA-2016-118913	6/25/2016	6/29/2016	Standard Class	AS-10240	Alan Shonely	Consumer	United States	Los Angeles	California	90049	West	OFF-AP-10000692	Office Suppli
2492	CA-2014-113579	12/13/2014	12/15/2014	Second Class	KD-16345	Katherine Ducich	Consumer	United States	Los Angeles	California	90049	West	OFF-PA-10001457	Office Suppli
2944	CA-2017-126242	11/19/2017	11/24/2017	Standard Class	MC-18100	Mick Crebagga	Consumer	United States	Los Angeles	California	90049	West	OFF-ST-10000675	Office Suppli
3534	CA-2014-110849	4/18/2014	4/23/2014	Standard Class	JL-15835	John Lee	Consumer	United States	Los Angeles	California	90049	West	OFF-AR-10002375	Office Suppli
3535	CA-2014-110849	4/18/2014	4/23/2014	Standard Class	JL-15835	John Lee	Consumer	United States	Los Angeles	California	90049	West	OFF-AR-10000657	Office Suppli
3536	CA-2014-110849	4/18/2014	4/23/2014	Standard Class	JL-15835	John Lee	Consumer	United States	Los Angeles	California	90049	West	OFF-FA-10000134	Office Suppli
3565	CA-2016-130029	7/3/2016	7/6/2016	First Class	GT-14755	Guy Thornton	Consumer	United States	Los Angeles	California	90049	West	OFF-PA-10000552	Office Suppli
3566	CA-2016-130029	7/3/2016	7/6/2016	First Class	GT-14755	Guy Thornton	Consumer	United States	Los Angeles	California	90049	West	OFF-FA-10001135	Office Suppli
3759	CA-2015-167745	9/18/2015	9/23/2015	Standard Class	GB-14530	George Bell	Corporate	United States	Los Angeles	California	90049	West	OFF-AR-10003602	Office Suppli
3760	CA-2015-167745	9/18/2015	9/23/2015	Standard Class	GB-14530	George Bell	Corporate	United States	Los Angeles	California	90049	West	OFF-SU-10004782	Office Suppli
3844	CA-2014-101931	10/28/2014	10/31/2014	First Class	TS-21370	Todd Sumrall	Corporate	United States	Los Angeles	California	90049	West	OFF-SU-10002301	Office Suppli
3845	CA-2014-101931	10/28/2014	10/31/2014	First Class	TS-21370	Todd Sumrall	Corporate	United States	Los Angeles	California	90049	West	OFF-SU-10000646	Office Suppli
3847	CA-2014-101931	10/28/2014	10/31/2014	First Class	TS-21370	Todd Sumrall	Corporate	United States	Los Angeles	California	90049	West	OFF-ST-10003442	Office Suppli
4339	CA-2015-125234	11/27/2015	12/1/2015	Standard Class	SN-20710	Steve Nguyen	Home Office	United States	Los Angeles	California	90049	West	OFF-PA-10000482	Office Suppli
4481	CA-2016-109365	11/3/2016	11/8/2016	Standard Class	XP-21865	Xylona Preis	Consumer	United States	Los Angeles	California	90049	West	OFF-FA-10001561	Office Suppli
4482	CA-2016-109365	11/3/2016	11/8/2016	Standard Class	XP-21865	Xylona Preis	Consumer	United States	Los Angeles	California	90049	West	OFF-AP-10002998	Office Suppli
4484	CA-2016-109365	11/3/2016	11/8/2016	Standard Class	XP-21865	Xylona Preis	Consumer	United States	Los Angeles	California	90049	West	OFF-PA-10001725	Office Suppli

Figure 5: Sample Superstore data loaded into Power BI.

Step 12: Check important fields such as Order ID, Order Date, Ship Date, Customer ID, Customer Name, Segment, Country, City, State, Postal Code, Region, Product ID, and Category.

Step 13: If filters or sorting are applied, verify that the displayed rows still match the expected data.

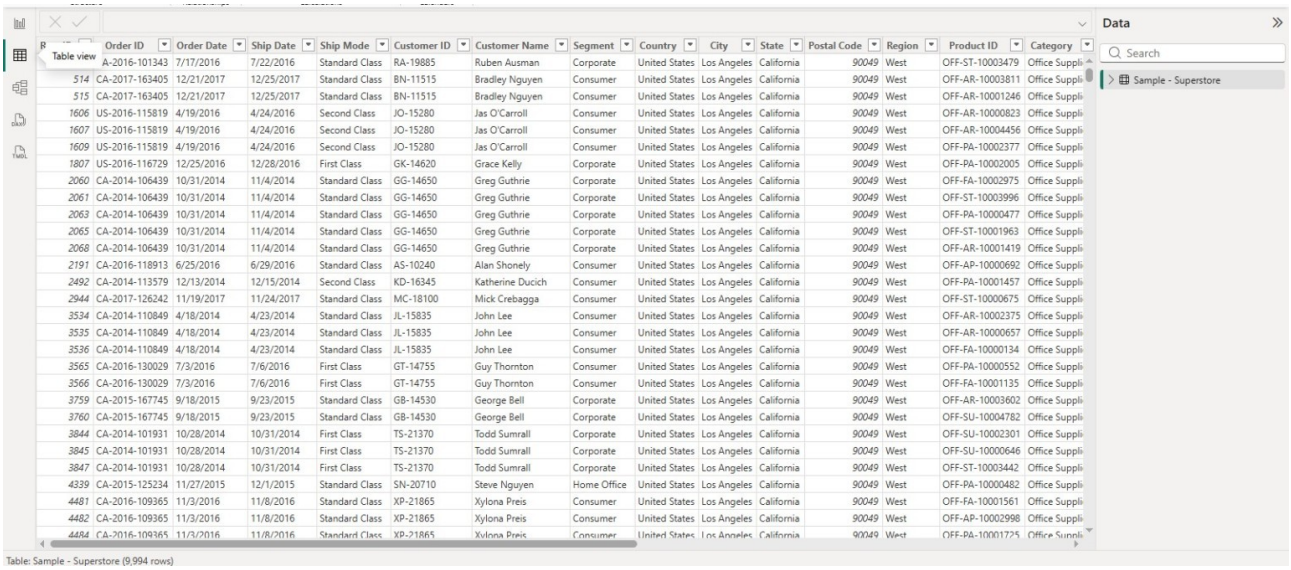


Figure 6: Example of the imported Superstore table viewed with a filtered/sorted set of rows.

8. Import Data Using an OData Feed

Power BI can also consume data exposed through an OData service. This is useful when the source is a web-accessible structured data service rather than a local file.

Step 14: In Power BI, select **Home → Get data → OData feed**.

Step 15: Enter the OData service address. The example shown uses the Northwind OData service endpoint:

http://services.odata.org/V3/Northwind/Northwind.svc/

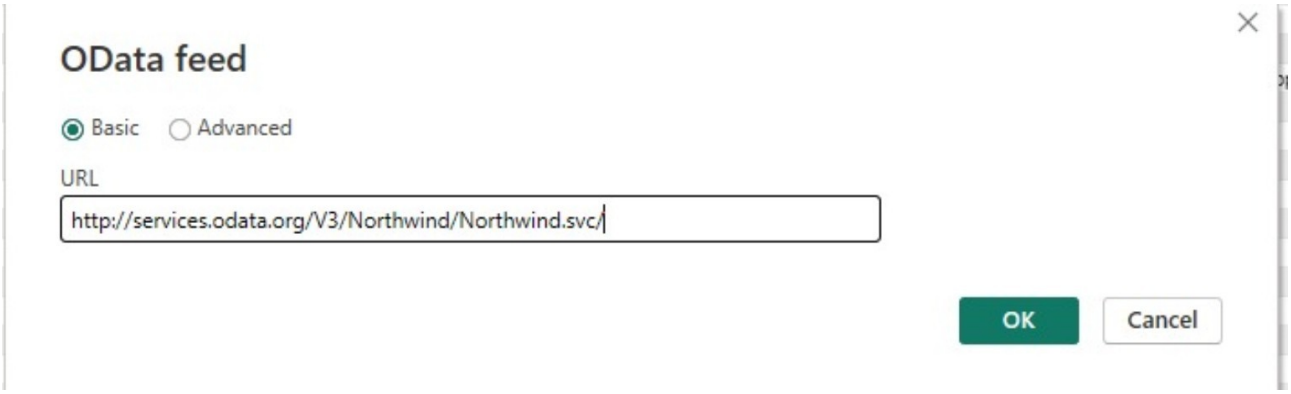


Figure 7: OData feed connector with the service URL entered.

Step 16: Select OK. Power BI retrieves the available entities/tables from the OData service.

9. Select Tables from the OData Navigator

Step 17: In the Navigator window, review the list of available tables.

Step 18: Select the required tables. In the example, tables such as Categories, Products, Orders, and Order Details are available.

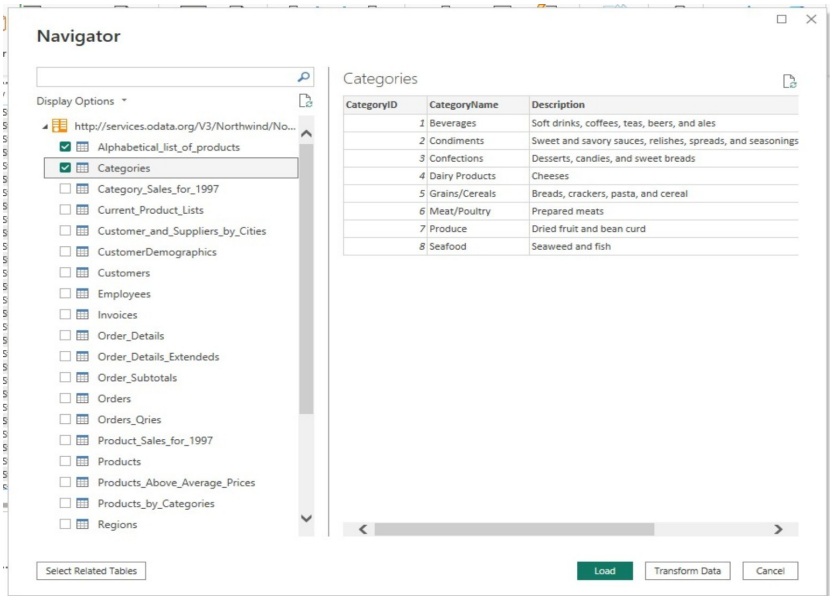


Figure 8: OData Navigator displaying available Northwind tables and a preview of Categories.

Step 19: Select **Load** to add the chosen tables to the Power BI data model. Select **Transform Data** if preprocessing is needed.

Step 20: After loading, use Table view and the Data/Fields pane to confirm that the selected tables and columns are available.

10. Verify OData Tables and Fields

Category	City	Country	Customer ID	Customer Name	Sum of Discount	Order Date	Order ID	Count of Postal Code	Product Name
Furniture	Akron	United States	EB-13705	Ed Braxton	0.40	10/20/2017	CA-2017-147277	1	Chromcraft Rectangular Conference Tables
Furniture	Akron	United States	TT-21070	Ted Trevino	0.20	5/18/2014	CA-2014-164224	1	Deflect-o Glass Clear Studded Chair Mats
Furniture	Alexandria	United States	AG-10495	Andrew Gjertsen	0.00	1/4/2015	CA-2015-104241	1	DAX Wood Document Frame
Furniture	Alexandria	United States	SD-20485	Shirley Daniels	0.00	1/26/2014	US-2014-155502	1	Eldon Image Series Black Desk Accessories
Furniture	Alexandria	United States	SD-20485	Shirley Daniels	0.00	1/26/2014	US-2014-155502	1	GE General Use Halogen Bulbs, 100 Watts, 1 Bulb per Pack
Furniture	Allen	United States	AG-10675	Anna Gayman	0.30	5/7/2015	CA-2015-137512	1	Chromcraft Round Conference Tables
Furniture	Allentown	United States	CJ-12010	Caroline Jumper	0.20	7/22/2016	CA-2016-122392	1	Master Caster Door Stop, Large Neon Orange
Furniture	Amarillo	United States	DS-13180	David Smith	0.32	3/31/2017	CA-2017-154907	1	Bush Mission Pointe Library
Furniture	Amarillo	United States	JE-15745	Joel Eaton	0.30	10/15/2015	CA-2015-139731	1	HON 5400 Series Task Chairs for Big and Tall
Furniture	Amarillo	United States	NM-18520	Neoma Murray	0.60	1/7/2016	CA-2016-158806	1	Executive Impressions 13" Clairmont Wall Clock
Furniture	Amarillo	United States	NR-18550	Nick Radford	0.30	5/2/2016	CA-2016-109400	1	Office Star - Contemporary Task Swivel chair with Loop Arms, C
Furniture	Anaheim	United States	BP-11185	Ben Peterman	0.00	12/29/2017	CA-2017-146626	1	Nu-Dell Executive Frame
Furniture	Anaheim	United States	EB-13705	Ed Braxton	0.20	6/14/2016	CA-2016-128223	1	Bush Cubix Conference Tables, Fully Assembled
Furniture	Anaheim	United States	KD-16615	Ken Dana	0.00	9/13/2015	US-2015-129007	1	Eldon Econocleat Chair Mats for Low Pile Carpets
Furniture	Anaheim	United States	KD-16615	Ken Dana	0.00	9/13/2015	US-2015-129007	1	Flat Face Poster Frame
Furniture	Anaheim	United States	KD-16615	Ken Dana	0.20	9/13/2015	US-2015-129007	1	Global Comet Stacking Armless Chair
Furniture	Anaheim	United States	KD-16615	Ken Dana	0.00	9/13/2015	US-2015-129007	1	Tensor Computer Mounted Lamp
Furniture	Anaheim	United States	WB-21850	William Brown	0.20	12/11/2016	CA-2016-165330	1	Global Wood Trimmed Manager's Task Chair, Khaki
Furniture	Anaheim	United States	WB-21850	William Brown	0.20	12/11/2016	CA-2016-165330	1	Hon Non-Folding Utility Tables
Furniture	Andover	United States	PS-18760	Pamela Stobb	0.00	3/9/2016	CA-2016-153269	1	Situations Contoured Folding Chairs, 4/Set
Furniture	Apopka	United States	CK-12205	Chloris Kastensmidt	0.20	7/28/2014	CA-2014-131541	1	DataProducts Ampli Magnifier Task Lamp, Black,
Furniture	Apple Valley	United States	AJ-10795	Anthony Johnson	0.15	7/17/2017	CA-2017-142342	1	Atlantic Metals Mobile 4-Shelf Bookcases, Custom Colors
Furniture	Apple Valley	United States	PK-19075	Pete Kriz	0.20	12/16/2016	CA-2016-118332	1	Office Star - Contemporary Swivel Chair with Padded Adjustab
Furniture	Arlington	United States	BF-11020	Barry Französisch	0.60	9/8/2017	US-2017-124779	1	DAX Black Cherry Wood-Tone Poster Frame
Furniture	Arlington	United States	BF-11020	Barry Französisch	0.30	9/8/2017	US-2017-124779	1	Global Armless Task Chair, Royal Blue
Furniture	Arlington	United States	CB-12025	Cassandra Brandow	0.30	8/24/2015	CA-2015-146563	1	Hon Racetrack Conference Tables
Furniture	Arlington	United States	KH-16510	Keith Herrera	0.00	6/18/2015	CA-2015-133242	1	Seth Thomas 8 1/2" Cubicle Clock
Furniture	Arlington	United States	LC-16960	Lindsay Castell	0.00	10/20/2016	CA-2016-105753	1	Aluminum Document Frame
Furniture	Arlington	United States	LE-16810	Laurel Elliston	0.00	5/19/2016	CA-2016-157245	1	Hon 4070 Series Pagoda Round Back Stacking Chairs
Total					1,561.09			9994	

Figure 9: Example of an OData table imported into Power BI.

CategoryID	CategoryName	Discontinued	ProductName	QuantityPerUnit	CategoryID	CategoryName	Description
1	Beverages	False	Lakkalikööri	500 ml	1	Beverages	Soft drinks, coffees, teas, beers, and ales
1	Beverages	False	Laughing Lumberjack Lager	24 - 12 oz bottles	1	Beverages	Soft drinks, coffees, teas, beers, and ales
1	Beverages	False	Outback Lager	24 - 355 ml bottles	1	Beverages	Soft drinks, coffees, teas, beers, and ales
1	Beverages	False	Rhönbräu Klosterbier	24 - 0.5 l bottles	1	Beverages	Soft drinks, coffees, teas, beers, and ales
1	Beverages	False	Sasquatch Ale	24 - 12 oz bottles	1	Beverages	Soft drinks, coffees, teas, beers, and ales
1	Beverages	False	Steeleye Stout	24 - 12 oz bottles	1	Beverages	Soft drinks, coffees, teas, beers, and ales
2	Condiments	False	Aniseed Syrup	12 - 550 ml bottles	2	Condiments	Sweet and savory sauces, relishes, spreads, and seasonings
2	Condiments	False	Chef Anton's Cajun Seasoning	48 - 6 oz jars	2	Condiments	Sweet and savory sauces, relishes, spreads, and seasonings
2	Condiments	False	Genen Shouyu	24 - 250 ml bottles	2	Condiments	Sweet and savory sauces, relishes, spreads, and seasonings
2	Condiments	False	Grandma's Boysenberry Spread	12 - 8 oz jars	2	Condiments	Sweet and savory sauces, relishes, spreads, and seasonings
2	Condiments	False	Gula Malacca	20 - 2 kg bags	2	Condiments	Sweet and savory sauces, relishes, spreads, and seasonings
2	Condiments	False	Louisiana Fiery Hot Pepper Sauce	32 - 8 oz bottles	2	Condiments	Sweet and savory sauces, relishes, spreads, and seasonings
2	Condiments	False	Louisiana Hot Spiced Okra	24 - 8 oz jars	2	Condiments	Sweet and savory sauces, relishes, spreads, and seasonings
2	Condiments	False	Northwoods Cranberry Sauce	12 - 12 oz jars	2	Condiments	Sweet and savory sauces, relishes, spreads, and seasonings
2	Condiments	False	Original Frankfurter grüne Soße	12 boxes	2	Condiments	Sweet and savory sauces, relishes, spreads, and seasonings
2	Condiments	False	Sirop d'érable	24 - 500 ml bottles	2	Condiments	Sweet and savory sauces, relishes, spreads, and seasonings
2	Condiments	False	Vegie-spread	15 - 625 g jars	2	Condiments	Sweet and savory sauces, relishes, spreads, and seasonings
3	Confections	False	Chocolade	10 pkgs.	3	Confections	Desserts, candies, and sweet breads
3	Confections	False	Gumbär Gummibärchen	100 - 250 g bags	3	Confections	Desserts, candies, and sweet breads
3	Confections	False	Maxilaku	24 - 50 g pkgs.	3	Confections	Desserts, candies, and sweet breads
3	Confections	False	NuNuCa Nuß-Nougat-Creme	20 - 450 g glasses	3	Confections	Desserts, candies, and sweet breads
3	Confections	False	Pavlova	32 - 500 g boxes	3	Confections	Desserts, candies, and sweet breads
3	Confections	False	Schoggi Schokolade	100 - 100 g pieces	3	Confections	Desserts, candies, and sweet breads
3	Confections	False	Scottish Longbreads	10 boxes x 8 pieces	3	Confections	Desserts, candies, and sweet breads
3	Confections	False	Sir Rodney's Marmalade	30 gift boxes	3	Confections	Desserts, candies, and sweet breads
3	Confections	False	Sir Rodney's Scones	24 pkgs. x 4 pieces	3	Confections	Desserts, candies, and sweet breads
3	Confections	False	Tarte au sucre	48 pies	3	Confections	Desserts, candies, and sweet breads
3	Confections	False	Teatime Chocolate Biscuits	10 boxes x 12 pieces	3	Confections	Desserts, candies, and sweet breads
3	Confections	False	Valkoinen suklaa	12 - 100 g bars	3	Confections	Desserts, candies, and sweet breads
3	Confections	False	Zaanse koeken	10 - 4 oz boxes	3	Confections	Desserts, candies, and sweet breads
4	Dairy Products	False	Camembert Pierrot	15 - 300 g rounds	4	Dairy Products	Cheeses
4	Dairy Products	False	Flotemysost	10 - 500 g pkgs.	4	Dairy Products	Cheeses

Figure 10: Example of product and category fields after importing related OData data.

11. Important Power BI Import Options

Option	Purpose
Load	Imports the selected data directly into the Power BI model.
Transform Data	Opens Power Query Editor for cleaning, shaping, filtering, and changing data types.
Navigator	Shows available tables/entities and a preview before importing.
Get Data	Provides access to Power BI's supported data connectors.
Refresh	Retrieves updated source data after the connection has been established.

12. Result

The Superstore CSV dataset was successfully **imported** into Power BI Desktop. The data was previewed and verified in Table view. The procedure also demonstrated how Power BI can connect to an **OData feed**, display available entities in **Navigator**, and **load** selected tables into the data model.

Practical – 2

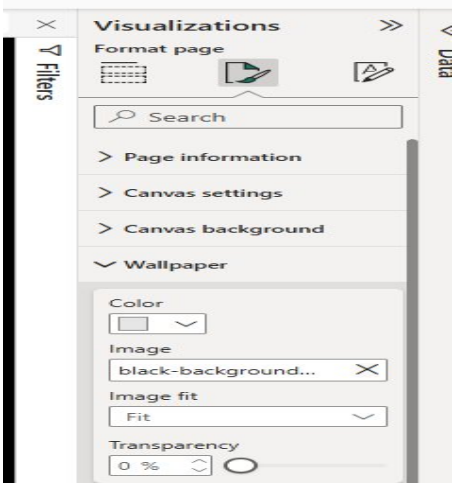
Aim:- Pie Chart and stacked column chart

Background Information:- Power BI is a data visualization tool used to analyze and present data. This practical uses a sales dataset to create Pie Chart and Stacked Column Chart visualizations for analyzing customer-wise quantity and product-wise delivery status.

Steps:-

1) Canvas Background image setup :-

- Click anywhere in canvas
- Go to visualization
- Click on Format your report page icon
- Go to wallpaper section select image you want to add to background
- Reduce the transparency



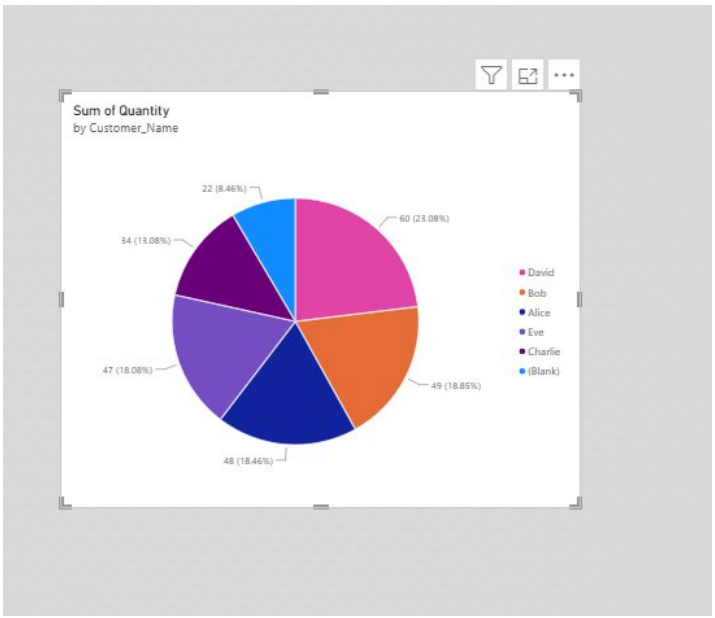
2) Visualization 2:- Sum of Quantity by Customer_Name:-

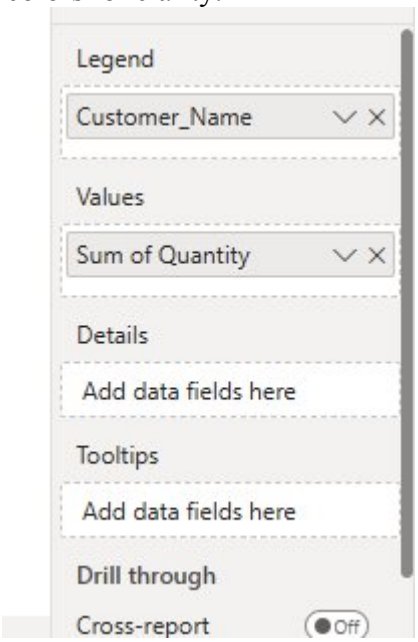
Data Used :- Customer name & Sum of Quantity.

Step 1: From the 'Visualizations' pane, select the Pie Chart.
Step 2: Drag the 'Customer_Name' field into the 'Legend' area.
Step 3: Drag the 'Quantity' field into the 'Values' area and set it to 'Sum'.
Step 4: The Pie Chart now shows the distribution of total quantities purchased by each customer.
Step 5: Labels and percentages are automatically displayed to show each customer's

contribution.

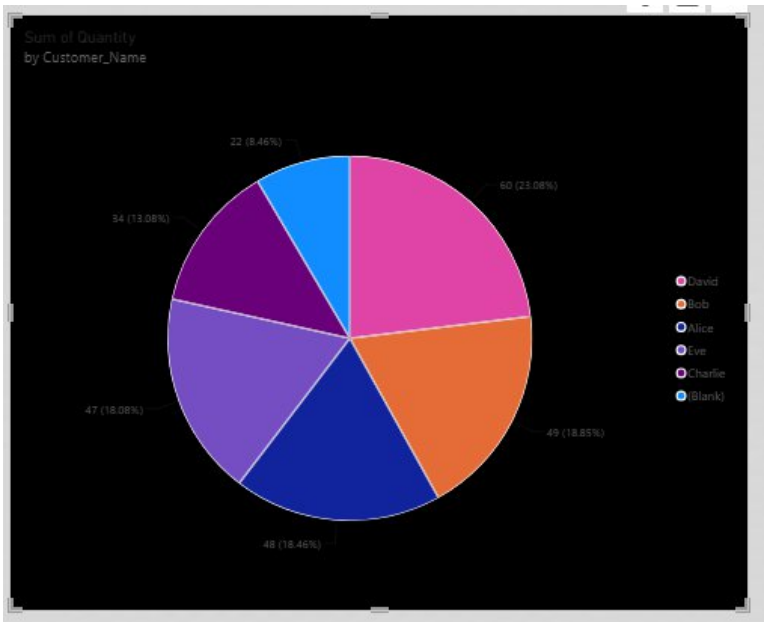
Step 6: Adjust formatting options like legend position and colors for clarity.

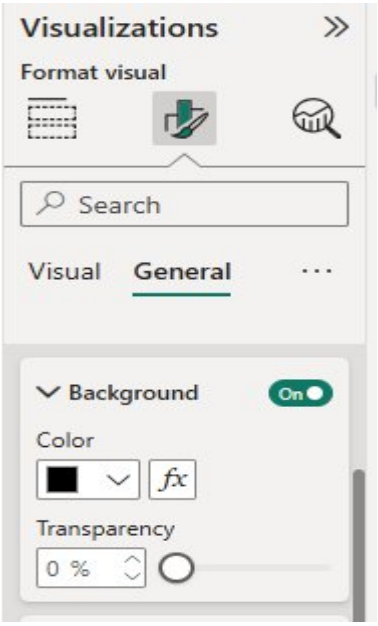




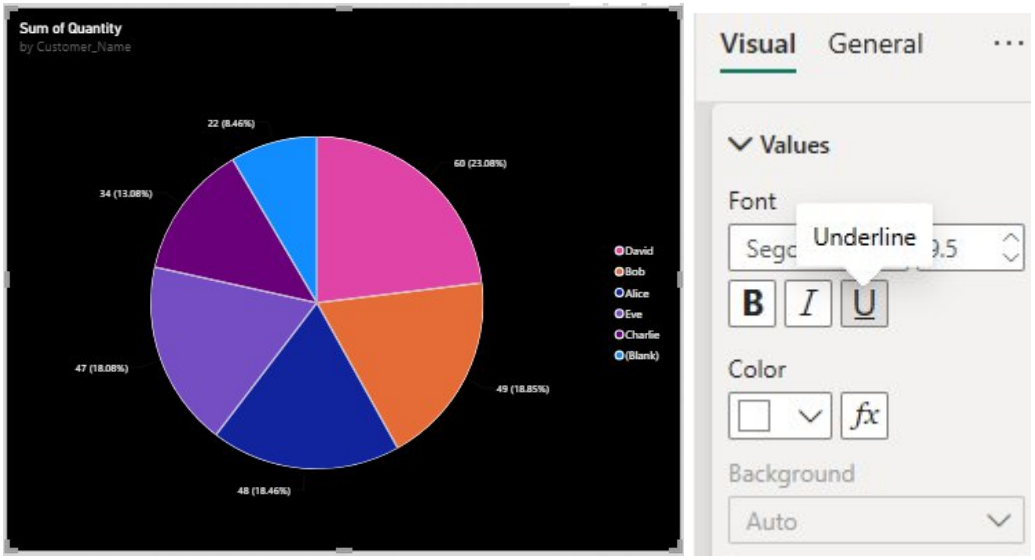
Step 7: - to make visualization align with background .

- Click on visual
- Go to visalization click on format your visual icon
- Then go to general click on Effect ☾ go to background ☾ set color similar to backrground ☾ reduce the Transparency





- Go to Title set Text color to white
- Then go to Visuals ☾ click on legend ☾ go to Text ☾ set color to white.
- Then go to Detailed Label ☾ go to Values ☾ set color to White.

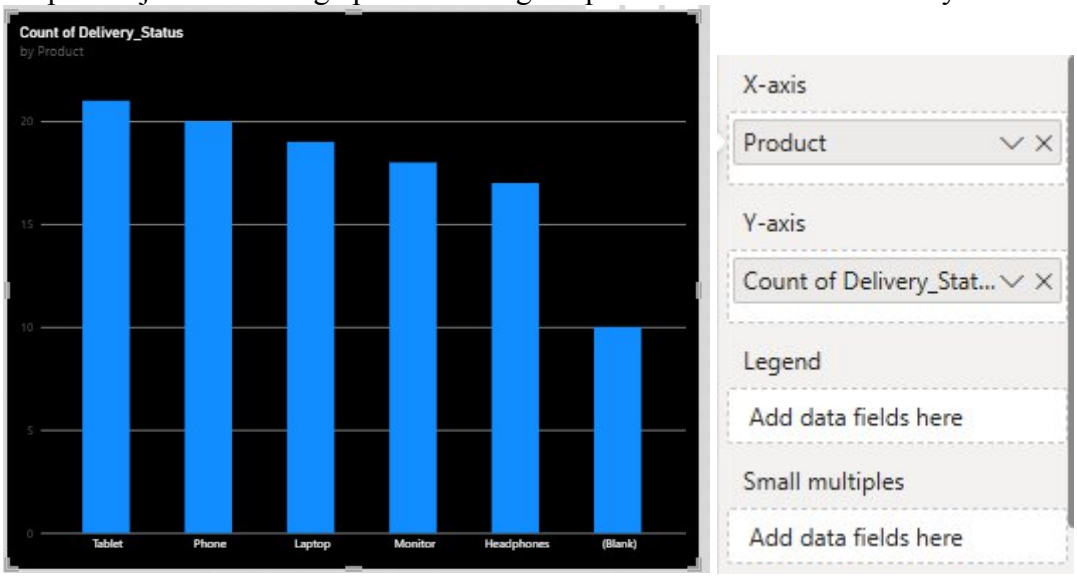


Visualization 3: - Stacked Column Chart.

Data Used :- Count of Delivery_Status by Product

Step 1: Load the dataset into Power BI by importing the 'Sales' table.
Step 2: From the 'Visualizations' panel, select the Stacked Column Chart.
Step 3: Drag the 'Product' field into the X-axis area.
Step 4: Drag the 'Delivery_Status' field into the Y-axis area and set it to 'Count'.
Step 5: The chart now displays the count of Delivery Status categorized by different products.

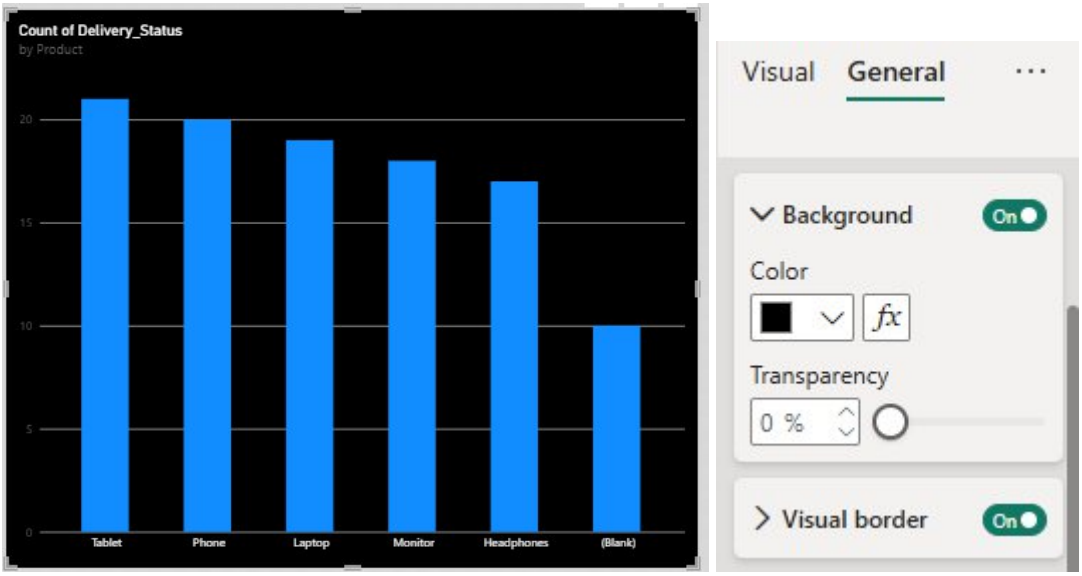
Step 6: Adjust formatting options like legend position and colors for clarity.



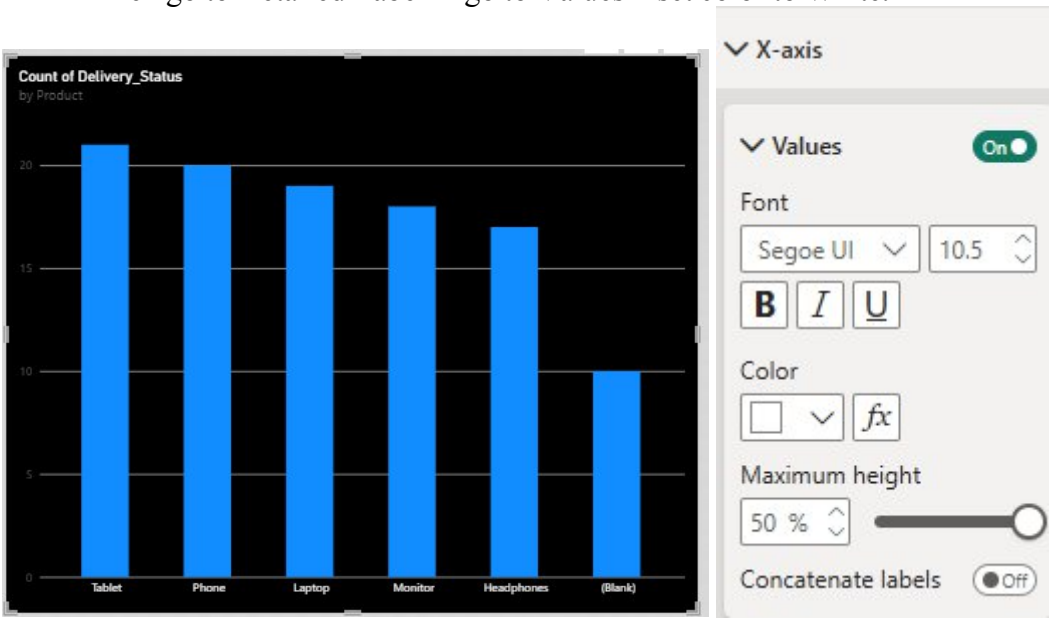
Step 7: - to make visualization align with background .

- Click on visual
- Go to visalization click on format your visual icon

- Then go to general click on Effect ☾ go to background ☾ set color similar to backrground ☾ reduce the Transparency



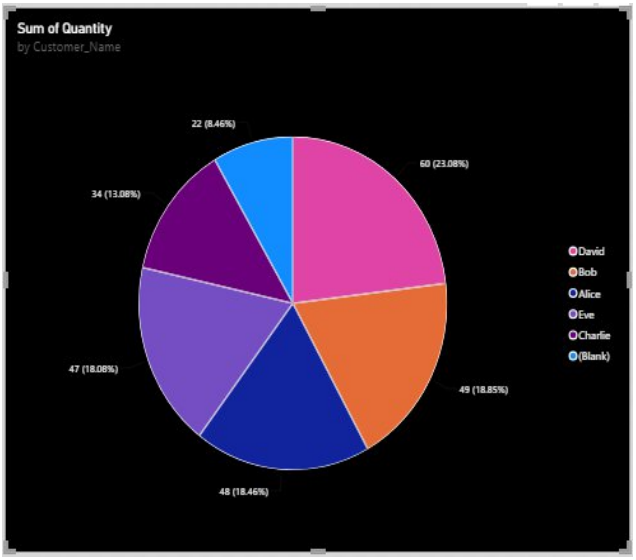
- Go to Title set Text color to white
- Then go to Visuals ☾ click on X-axis ☾ go to Title And Value ☾ set color to white.
- Then go to Visuals ☾ click on Y-axis ☾ go to Title And Value ☾ set color to white.
- Then go to Detailed Label ☾ go to Values ☾ set color to White.

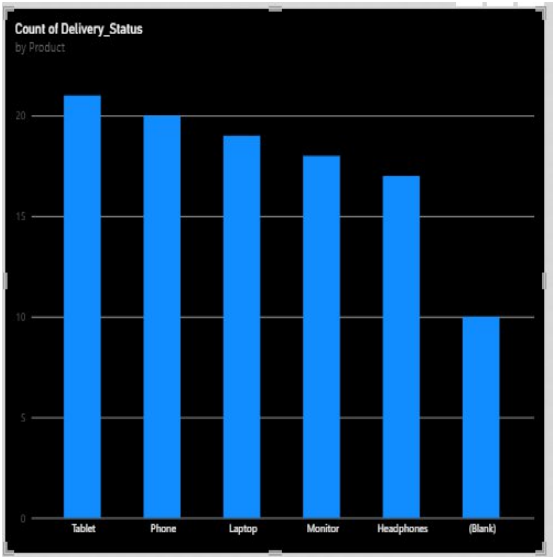


Conclusion

Through these two visualizations, we analyzed product-wise delivery status and customer-wise quantity distribution. Power BI made it easy to visualize the data, helping in better understanding of sales performance and customer contributions.

Here is the final Representation of both the Visualization.


Sum of Quantity
by Customer_Name
22 (8.46%)
60 (23.08%)
34 (13.08%)
47 (18.08%)
49 (18.85%)
48 (18.46%)
David
Bob
Alice
Eve
Charlie
(Blank)


Count of Delivery_Status
by Product
20
15
10
5
0
Tablet
Phone
Laptop
Monitor
Headphones
(Blank)

Practical:- 2 (B)

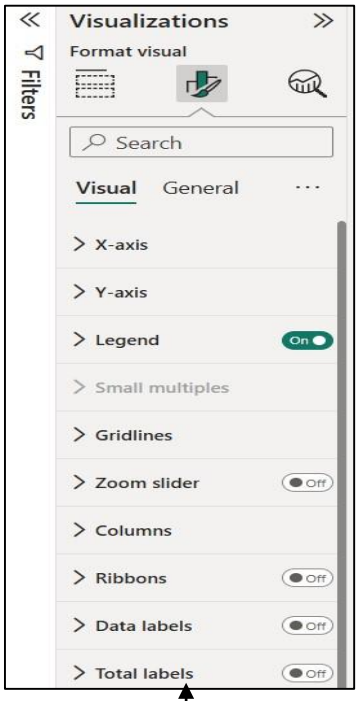
Aim :- Perform ETL, Create -Stacked Column chart.

The **dataset** used contains sales-related fields :-

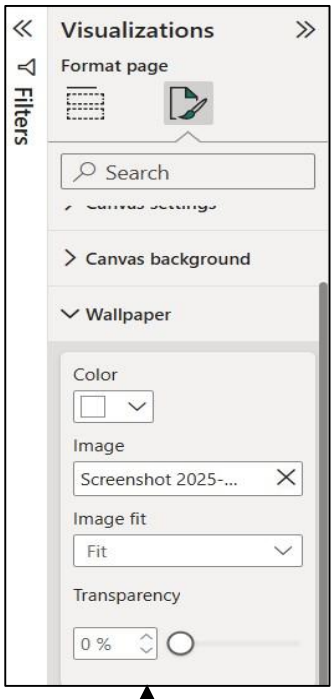
Customer_Name , Product , Quantity ,and Delivery_Status.

Report Customization:-

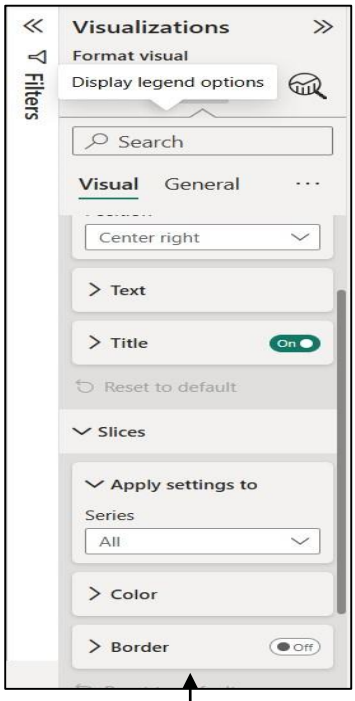
- **Background:** Change from white to any custom image.
  - Click on report background → go to Format → Wallpaper → Image → Browse → Fit.
- **Chart Styling:**
  - Customize font, colors, background, and transparency.
  - Under Visuals → X-axis / Y-axis settings, adjust font size, color, and formatting.
- Similar customization is available for Pie Chart (though some options may differ).



For Report's Background



For Stacked Column Chart



For Pie Chart

1. Visualization 1:- Stacked Column chart .

Data Used :- Count of product & Delivery Status.

Step 1: From the 'Visualizations' pane, select the Stacked Column chart.

Step 2: Drag the **Count of product** field into the 'X-axis' area.

Step 3: Drag the **Delivery Status** field into the 'Values' area and set it to 'Sum'.

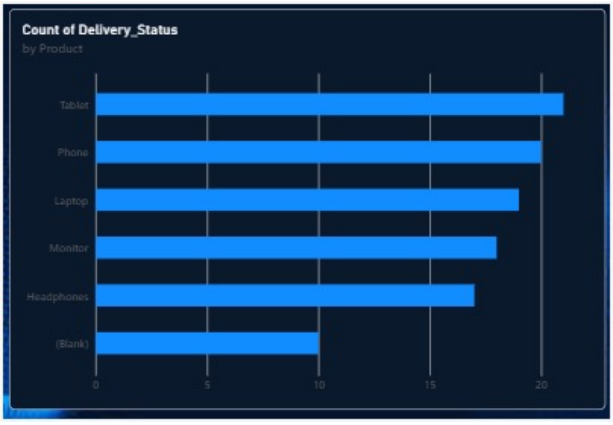
Step 4: The chart now shows the distribution of Count of product & Delivery Status.

Step 5: Adjust formatting options like axis title, colors, and labels for clarity.

Step 6: To align with the background, adjust visual background transparency and set text color to white.

Step 7**: -** to make visualization align with background .

- Click on visual
- Go to visalization click on format your visual icon
- Then go to general click on Effect ❼ go to background❼set color similar to backrground ❼reduce the Transparency
- Go to Title set Text color to white
- Then go to Visuals ❼click on legend ❼go to Text❼ set color to white.
- Then go to Detailed Label❼ go to Values❼set color to White.



2. Visualization 2: - Skatered Column Chart.

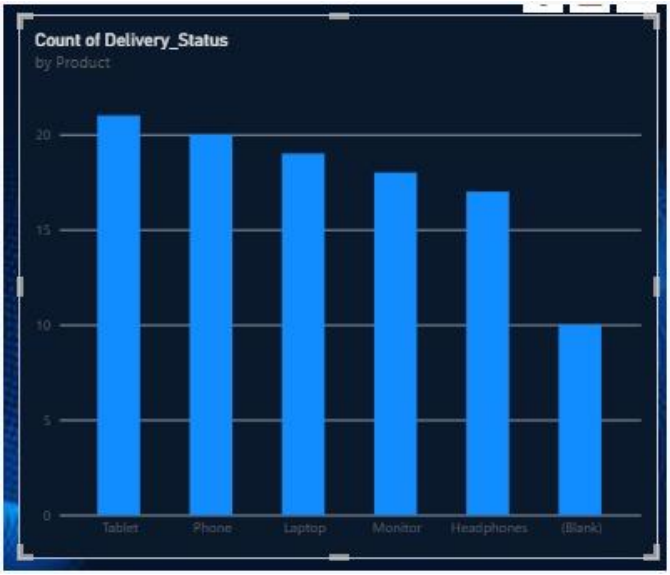
Data Used :- Count of Delivery_Status by Product

Step 1: Load the dataset into Power BI by importing the 'Sales' table.
Step 2: From the 'Visualizations' panel, select the Skatered Column Chart.
Step 3: Drag the 'Product' field into the X-axis area.
Step 4: Drag the 'Delivery_Status' field into the Y-axis area and set it to 'Count'.
Step 5: The chart now displays the count of Delivery Status categorized by different products.

Step 6: Adjust formatting options like legend position and colors for clarity.

Step 7: - to make visualization align with background

- Click on visual
- Go to visalization click on format your visual icon
- Then go to general click on Effect ❼ go to background❼set color similar to backrground ❼reduce the Transparency.
- Go to Title set Text color to white
- Then go to Visuals ❼click on X-axis ❼go to Title And Value❼ set color to white.
- Then go to Visuals ❼click on Y-axis ❼go to Title And Value❼ set color to white.
- Then go to Detailed Label❼ go to Values❼set color to White.



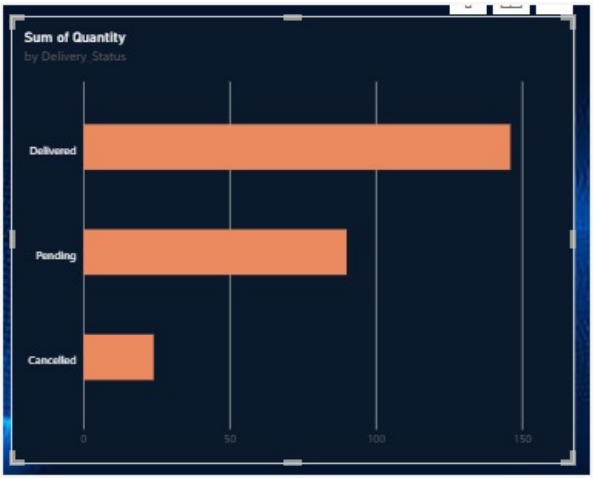
3.Visualization 3: - Clustered Bar Chart.

Data Used :- Sum of Quantity by Delivery_Status.

Step 1: Load the dataset into Power BI by importing the 'Sales' table.
Step 2: From the 'Visualizations' panel, select the Clustered Bar Chart.
Step 3: Drag the 'Quantity' field into the X-axis area.
Step 4: Drag the 'Delivery_Status' field into the Y-axis area and set it to 'Count'.
Step 5: The chart now displays the count of Delivery Status categorized by different products Quantity.

Step 6: Adjust formatting options like legend position and colors for clarity.

Step 7: - to make visualization align with background.



4.Visualization 4: - Clustered Column Chart.

Data Used :- Sum of Quantity by Delivery_Status.

Step 1: Load the dataset into Power BI by importing the 'Sales' table.
Step 2: From the 'Visualizations' panel, select the Clustered Bar Chart.
Step 3: Drag the 'Quantity' field into the X-axis area.
Step 4: Drag the 'Delivery_Status' field into the Y-axis area and set it to 'Count'.
Step 5: The chart now displays the count of Delivery Status categorized by different products Quantity.

Step 6: Adjust formatting options like legend position and colors for clarity.

Step 7: - to make visualization align with background.



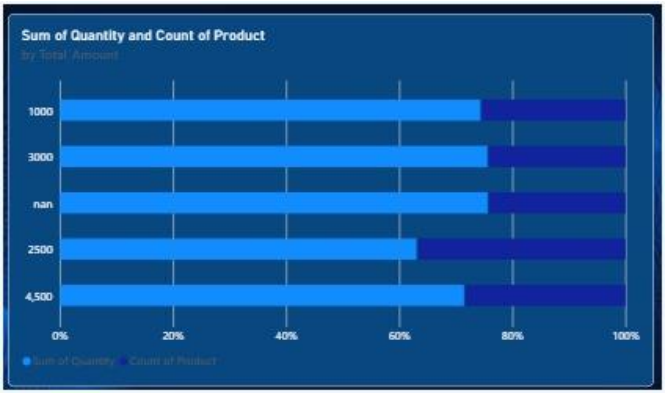
5. Visualization 5: - 100% Stacked Bar Chart.

Data Used :- Sum of Quantity and count of product by total Amount.

Step 1: Load the dataset into Power BI by importing the 'Sales' table.
Step 2: From the 'Visualizations' panel, select the Clustered Bar Chart.
Step 3: Drag the ' Sum of Quantity and count of product ' field into the X-axis area.
Step 4: Drag the ‘ total Amount ' field into the Y-axis area and set it to 'Count'.
Step 5: The chart now displays the count of Delivery Status categorized by different products Quantity.

Step 6: Adjust formatting options like legend position and colors for clarity.

Step 7: - to make visualization align with background.



Final_Output:-



Conclusion:-

Through these Five visualizations, we analyzed product-wise delivery status and customerwise quantity distribution along with Sum of Quantity and count of product by total Amount Power BI made it easy to visualize the data, helping in better understanding of sales performance and customer contributions.



PRACTICAL NO. 4

Aim: To perform data classification using classification algorithms (Decision Tree, K-Nearest Neighbours and Naive Bayes) in Python and evaluate their performance.

Objectives:

- Load and explore a labelled dataset.
- Split data into training and testing sets.
- Build Decision Tree, KNN and Naive Bayes classifiers.
- Evaluate the models using accuracy, confusion matrix and classification report.

Background Information:

Classification is a supervised data mining technique that predicts a categorical class label for new records after learning from labelled training data. It is related to classification problems, evaluation of classification models and Bayesian methods.

A Decision Tree splits data using attribute tests. K-Nearest Neighbours (KNN) assigns the majority class among the k closest records. Naive Bayes applies Bayes' theorem while assuming that the attributes are independent. The models are evaluated on unseen test data. Accuracy is the proportion of correct predictions. A confusion matrix shows correct and incorrect predictions for each class, while precision, recall and F1-score describe class-wise performance.

Dataset:

Dataset	File / Link	Use
Iris flower dataset (150 rows, 3 classes)	https://raw.githubusercontent.com/mwaskom/seaborn-data/master/iris.csv (original: https://archive.ics.uci.edu/dataset/53/iris)	Used in this practical
Breast Cancer Wisconsin (optional, 2 classes)	https://archive.ics.uci.edu/dataset/17/breast-cancer-wisconsin-diagnostic or sklearn.datasets.load_breast_cancer()	For extra practice

Procedure / Steps:

Step 1: Import Libraries and Load the Dataset

Open a new Google Colab notebook and run the following cell to import the required libraries, load the Iris dataset and inspect the first rows and species counts.

```
import pandas as pd, numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import (
    accuracy_score, confusion_matrix, classification_report,
    ConfusionMatrixDisplay
```

```
)

url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/iris.csv"
df = pd.read_csv(url)
print(df.head())
print(df["species"].value_counts())
```

Expected output / observation: The first five rows are displayed. Each of the three species has 50 records.

```
...     sepal_length  sepal_width  petal_length  petal_width species
    0           5.1          3.5           1.4          0.2  setosa
    1           4.9          3.0           1.4          0.2  setosa
    2           4.7          3.2           1.3          0.2  setosa
    3           4.6          3.1           1.5          0.2  setosa
    4           5.0          3.6           1.4          0.2  setosa
    species
    setosa        50
    versicolor    50
    virginica     50
    Name: count, dtype: int64
```

Step 2: Prepare Features and Split the Data

Separate the input features (X) from the target class label (y). Split the dataset into 80% training data and 20% testing data. The stratify argument preserves class proportions.

```
X = df.drop(columns="species")
y = df["species"]

X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=42, stratify=y
)
print("Training set:", X_train.shape, " Testing set:", X_test.shape)
```

Expected output / observation: The split creates 120 training rows and 30 testing rows.

```
...  Training set: (120, 4)  Testing set: (30, 4)
```

Step 3: Decision Tree Classifier

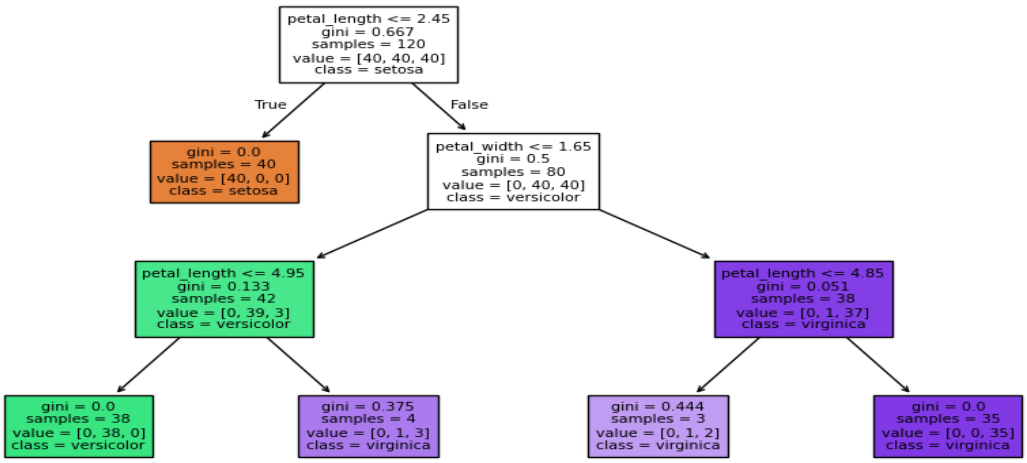
Train a Decision Tree classifier, calculate its accuracy on the test data and display the tree. A maximum depth of 3 keeps the tree small and helps avoid overfitting.

```
tree = DecisionTreeClassifier(max_depth=3, random_state=42)
tree.fit(X_train, y_train)
pred_tree = tree.predict(X_test)
acc_tree = accuracy_score(y_test, pred_tree)
print("Decision Tree accuracy:", round(acc_tree, 3))

plt.figure(figsize=(11, 6))
plot_tree(
   tree, feature_names=X.columns,
   class_names=list(tree.classes_), filled=True, fontsize=8
)
plt.show()
```

Expected output / observation: The accuracy is printed and the tree displays splits based on features such as petal length and petal width.

Decision Tree accuracy: 0.967.



Step 4: K-Nearest Neighbours (KNN) Classifier

KNN uses distances to identify nearby records, so scale the features before fitting the model. Use five neighbours for classification.

```
scaler = StandardScaler().fit(X_train)
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(scaler.transform(X_train), y_train)
pred_knn = knn.predict(scaler.transform(X_test))
acc_knn = accuracy_score(y_test, pred_knn)
print("KNN accuracy:", round(acc_knn, 3))
```

Expected output / observation: The KNN accuracy on the test data is printed.

```
KNN accuracy: 0.933
```

Step 5: Naive Bayes Classifier

Train the Gaussian Naive Bayes model and calculate its test accuracy.

```
nb = GaussianNB()
nb.fit(X_train, y_train)
pred_nb = nb.predict(X_test)
acc_nb = accuracy_score(y_test, pred_nb)
print("Naive Bayes accuracy:", round(acc_nb, 3))
```

Expected output / observation: The Naive Bayes accuracy on the test data is printed.

```
Naive Bayes accuracy: 0.967
```

Step 6: Compare the Models

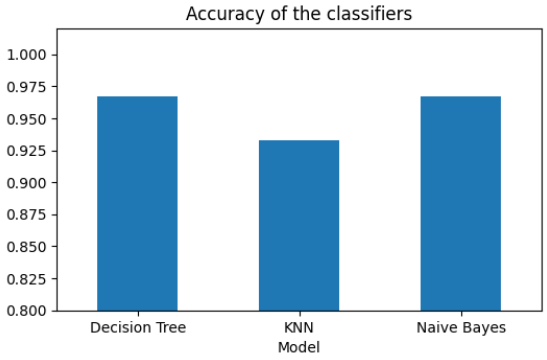
Collect the accuracy values for Decision Tree, KNN and Naive Bayes in a table. Draw a bar chart to compare their performance.

```
results = pd.DataFrame({
    "Model": ["Decision Tree", "KNN", "Naive Bayes"],
    "Accuracy": [acc_tree, acc_knn, acc_nb]
}).round(3)
print(results)

results.plot(
    x="Model", y="Accuracy", kind="bar",
    ylim=(0.8, 1.02), legend=False,
    figsize=(6, 3.5), rot=0
)
plt.title("Accuracy of the classifiers")
plt.show()
```

Expected output / observation: The comparison table and bar chart show the performance of all three models on this train/test split.

```
...            Model  Accuracy
0  Decision Tree     0.967
1            KNN     0.933
2    Naive Bayes     0.967
```



Step 7: Confusion Matrix and Classification Report

Evaluate the KNN model in more detail using a confusion matrix and classification report.

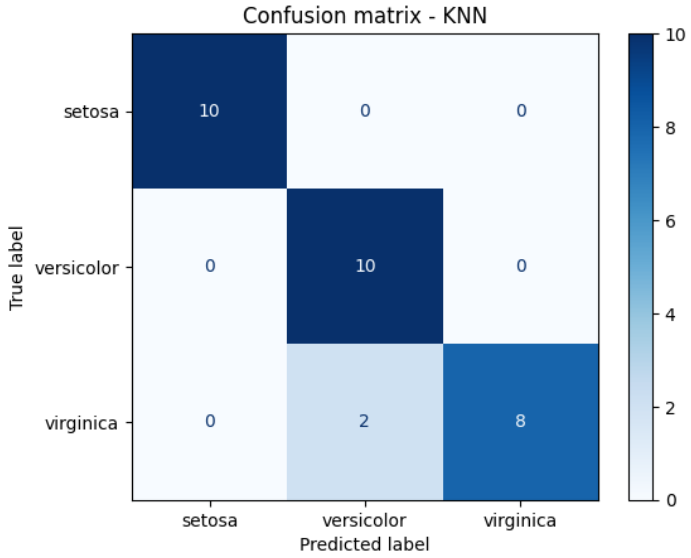
```
print(confusion_matrix(y_test, pred_knn))
print(classification_report(y_test, pred_knn))
ConfusionMatrixDisplay.from_predictions(y_test, pred_knn, cmap="Blues")
plt.title("Confusion matrix - KNN")
plt.show()
```

Expected output / observation: The confusion matrix shows correct predictions along its diagonal. The classification report provides precision, recall and F1-score for each species.

```
[[10  0  0]
 [ 0 10  0]
 [ 0  2  8]]
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       0.83      1.00      0.91        10
   virginica       1.00      0.80      0.89        10

    accuracy                           0.93        30
   macro avg       0.94      0.93      0.93        30
weighted avg       0.94      0.93      0.93        30
```



Step 8: Predict a New Record

Create a new flower record using the four measurements below and use the trained KNN model to predict its species.

```
new = pd.DataFrame([[5.9, 3.0, 5.1, 1.8]], columns=X.columns)
print("Predicted species:", knn.predict(scaler.transform(new))[0])
```

Expected output / observation: The predicted species for the new flower measurements is printed.

```
Predicted species: virginica
```

Result: Classification models were built and evaluated using Decision Tree, KNN and Naive Bayes algorithms and the species of a new record was predicted.

PRACTICAL NO. 5

Aim: To perform customer segmentation using the K-Means and Hierarchical clustering algorithms in Python.

Objectives:

- Load and scale customer data.
- Find the best number of clusters using the elbow method and silhouette score.
- Build K-Means and hierarchical clusters.
- Visualize and interpret the customer segments.

Background Information:

Clustering is an unsupervised data mining technique that groups similar records without using class labels. It is associated with clustering methods, partition methods, hierarchical methods and evaluation of clustering models.

K-Means is a partition method: k centroids are chosen, every record is assigned to the nearest centroid, and the centroids are recalculated until they stop changing. Hierarchical (agglomerative) clustering repeatedly merges the closest clusters and displays the result as a dendrogram.

The number of clusters can be selected using the elbow method, which examines inertia against k, and evaluated using the silhouette score. The silhouette score ranges from -1 to +1, with higher values indicating better-defined clusters. Clustering is widely used for relational marketing and customer segmentation.

Dataset:

Dataset	File / Link	Use
Mall Customers (200 rows)	https://raw.githubusercontent.com/miltonsuggs/11-mall-customer-segmentation/main/Mall_Customers.csv (original on Kaggle: https://www.kaggle.com/datasets/vjchoudhary7/customer-segmentation-tutorial-in-python)	Used in this practical

Procedure / Steps:

Step 1: Load the Dataset

Open Google Colab and run the following cell to import the required libraries, load the Mall Customers dataset, rename two columns for convenience and inspect the records and summary statistics.

```
import pandas as pd, numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score, adjusted_rand_score
from scipy.cluster.hierarchy import linkage, dendrogram
```

```
url = "https://raw.githubusercontent.com/miltonsuggs/11-mall-customer-segmentation/main/Mall_Customers.csv"
df = pd.read_csv(url).rename(columns={
    "Annual Income (k$)": "Income",
    "Spending Score (1-100)": "Score"
})
print(df.head())
print(df[["Age", "Income", "Score"]].describe().round(1))
```

Expected output / observation: The dataset displays 200 customers with Gender, Age, Income and Spending Score.

```
   CustomerID  Gender  Age  Income  Score
0           1    Male   19      15     39
1           2    Male   21      15     81
2           3  Female   20      16      6
3           4  Female   23      16     77
4           5  Female   31      17     40
              Age  Income  Score
count       200.0   200.0  200.0
mean         38.8    60.6   50.2
std          14.0    26.3   25.8
min          18.0    15.0    1.0
25%          28.8    41.5   34.8
50%          36.0    61.5   50.0
75%          49.0    78.0   73.0
max          70.0   137.0   99.0
```

Step 2: Select and Scale the Features

Use Annual Income and Spending Score as the clustering features. Standard scaling gives both features a comparable scale.

```
features = ["Income", "Score"]
X = StandardScaler().fit_transform(df[features])
print("Scaled data shape:", X.shape)
```

Expected output / observation: The scaled array X has 200 rows and 2 columns.

```
Scaled data shape: (200, 2)
```

Step 3: Find k using the Elbow Method and Silhouette Score

Run K-Means for k values from 2 to 10. Record inertia and silhouette scores, then plot both measures to help select the number of clusters.

```
 inertia, sil = [], []
ks = range(2, 11)
for k in ks:
    km = KMeans(n_clusters=k, n_init=10, random_state=42).fit(X)
    inertia.append(km.inertia_)
    sil.append(silhouette_score(X, km.labels_))

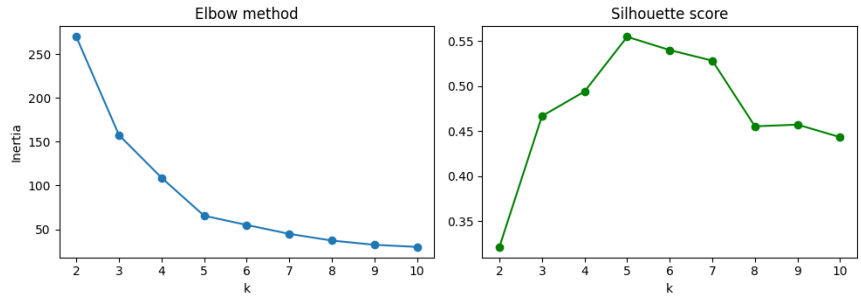
print(pd.DataFrame({
    "k": list(ks),
    "inertia": np.round(inertia, 1),
    "silhouette": np.round(sil, 3)
}))
fig, ax = plt.subplots(1, 2, figsize=(10, 3.6))
ax[0].plot(ks, inertia, marker="o")
ax[0].set_title("Elbow method")
```

```
ax[0].set_xlabel("k")
ax[0].set_ylabel("Inertia")
ax[1].plot(ks, sil, marker="o", color="green")
ax[1].set_title("Silhouette score")
ax[1].set_xlabel("k")

plt.tight_layout()
plt.show()
```

Expected output / observation: Inertia is the sum of squared distances to the nearest centroid. In the handbook, the inertia curve bends at k = 5 and the silhouette score is highest near k = 5.

```
    k  inertia  silhouette
0   2    269.7       0.321
1   3    157.7       0.467
2   4    108.9       0.494
3   5     65.6       0.555
4   6     55.1       0.540
5   7     44.9       0.528
6   8     37.2       0.455
7   9     32.4       0.457
8  10     30.0       0.443
```



Step 4: Build the K-Means Model (k = 5)

Fit the final K-Means model with five clusters, assign each customer a cluster number and display cluster sizes and the silhouette score.

```
kmeans = KMeans(n_clusters=5, n_init=10, random_state=42)
df["Cluster"] = kmeans.fit_predict(X)
print(df["Cluster"].value_counts().sort_index())
print("Silhouette score:", round(silhouette_score(X, df["Cluster"]), 3))
```

Expected output / observation: Each customer is assigned to one of five clusters, and the cluster sizes are printed.

```
Cluster
0    81
1    39
2    22
3    35
4    23
Name: count, dtype: int64
Silhouette score: 0.555
```

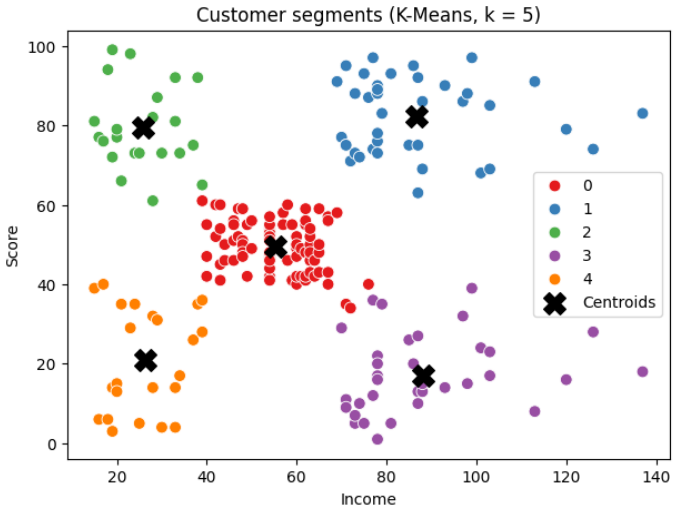
Step 5: Visualize the Clusters

Plot Income against Spending Score, colour the points by cluster and mark the cluster centroids.

```
centers = StandardScaler().fit(df[features]).inverse_transform(
    kmeans.cluster_centers_
)
plt.figure(figsize=(7, 5))
sns.scatterplot(
    data=df, x="Income", y="Score",
    hue="Cluster", palette="Set1", s=60
)


plt.scatter(
    centers[:, 0], centers[:, 1],
    c="black", marker="X", s=200, label="Centroids"
)
plt.title("Customer segments (K-Means, k = 5)")
plt.legend()
plt.show()
```

Expected output / observation: The scatter plot shows five customer groups, with an X marking each centroid.



Step 6: Profile the Segments

Calculate the average Age, Income and Spending Score for each cluster, along with the number of customers. Use these values to interpret and name the segments.

```
profile = df.groupby("Cluster")[["Age", "Income", "Score"]].mean().round(1)
profile["Customers"] = df["Cluster"].value_counts().sort_index()
print(profile)
```

Expected output / observation: The profile table can reveal groups such as high-income/high-spending customers and high-income/low-spending customers. Naming each segment is the business interpretation step.

Cluster	Age	Income	Score	Customers
0	42.7	55.3	49.5	81
1	32.7	86.5	82.1	39
2	25.3	25.7	79.4	22
3	41.1	88.2	17.1	35
4	45.2	26.3	20.9	23

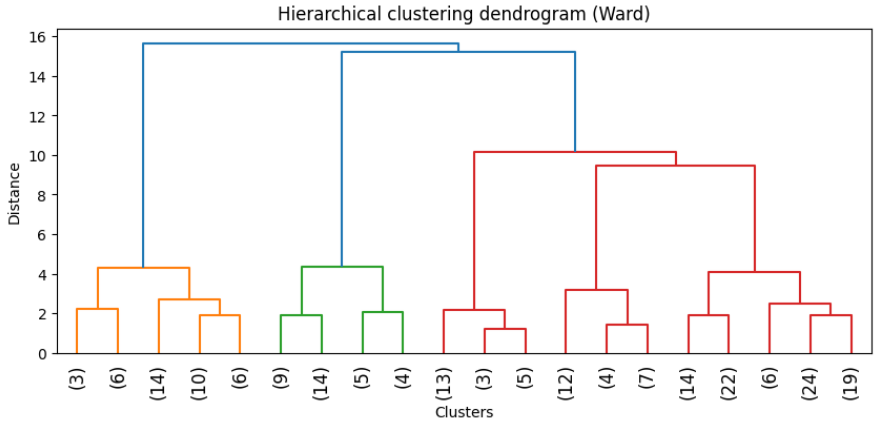
Step 7: Hierarchical Clustering

Create a dendrogram using Ward linkage and compare hierarchical clustering with the K-Means result using the adjusted Rand index.

```
Z = linkage(X, method="ward")
plt.figure(figsize=(10, 4))
dendrogram(Z, truncate_mode="lastp", p=20, leaf_rotation=90)
plt.title("Hierarchical clustering dendrogram (Ward)")
plt.xlabel("Clusters")
plt.ylabel("Distance")
plt.show()

hier = AgglomerativeClustering(
    n_clusters=5, linkage="ward"
).fit_predict(X)
print(
    "Agreement with K-Means (adjusted Rand index):",
    round(adjusted_rand_score(df["Cluster"], hier), 3)
)
```

Expected output / observation: The dendrogram displays the cluster-merging structure. The adjusted Rand index indicates how closely the hierarchical and K-Means assignments agree.




```
Agreement with K-Means (adjusted Rand index): 0.942
```

Result: Customers were segmented into five groups using K-Means and hierarchical clustering, and the groups were interpreted using their average age, income and spending score

Practical No. 6
Linear Regression on Data Warehouse Data (Python)

Complete practical with actual output screenshots

Aim

To perform simple and multiple linear regression on weekly store sales data from a data warehouse using Python, and interpret the results.

Software / Tools Used

Python 3, Google Colab, pandas, scikit-learn, statsmodels, matplotlib and seaborn.

Objectives

- Explore the relationship between sales and its drivers.
- Build simple and multiple linear regression models.
- Evaluate the models using R-squared, MAE and RMSE.
- Interpret coefficients and use the model for prediction.

Theory

Regression is a data mining technique that predicts a continuous numeric target variable from one or more input variables. Simple linear regression fits y = b0 + b1x, while multiple linear regression uses several predictors. The coefficients are estimated by the least squares method.

R-squared shows the share of variation in the target explained by the model. MAE and RMSE measure prediction error in the target units. Residuals are actual minus predicted values and should be reasonably scattered around zero.

Dataset

The dataset contains 500 store-week records with the same core variables used by the original practical: store, ad_spend_k, footfall, avg_discount_pct and weekly_sales_k.

Column	Description	Type
store	Store name	Categorical
ad_spend_k	Advertising spend (thousand Rs.)	Numeric
footfall	Weekly store visitors	Numeric
avg_discount_pct	Average discount percentage	Numeric
weekly_sales_k	Weekly sales (thousand Rs.)	Numeric target

Step 1: Load and Explore the Data

```
import pandas as pd, numpy as np
import matplotlib.pyplot as plt, seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
import statsmodels.api as sm

df = pd.read_csv("store_sales_regression.csv")
```

```
print(df.head())
print(df.describe().round(1))
print("Missing values:", df.isna().sum().sum())
```

Step 1: Load and Explore the Data

Python • Google Colab cell

```
df = pd.read_csv("store_sales_regression.csv")
print(df.head())
print(df.describe().round(1))
print("Missing values:", df.isna().sum().sum())
```

Output

```
             store  ad_spend_k  footfall  avg_discount_pct  weekly_sales_k
0  Hyderabad Store        21.2      2770               4.6           384.6
1    Kolkata Store        26.2       997               4.3           347.1
2    Chennai Store        51.8      3426               0.6           507.7
3  Hyderabad Store        22.4      2002              13.0           357.8
4      Delhi Store        14.3      2835               7.4           352.9

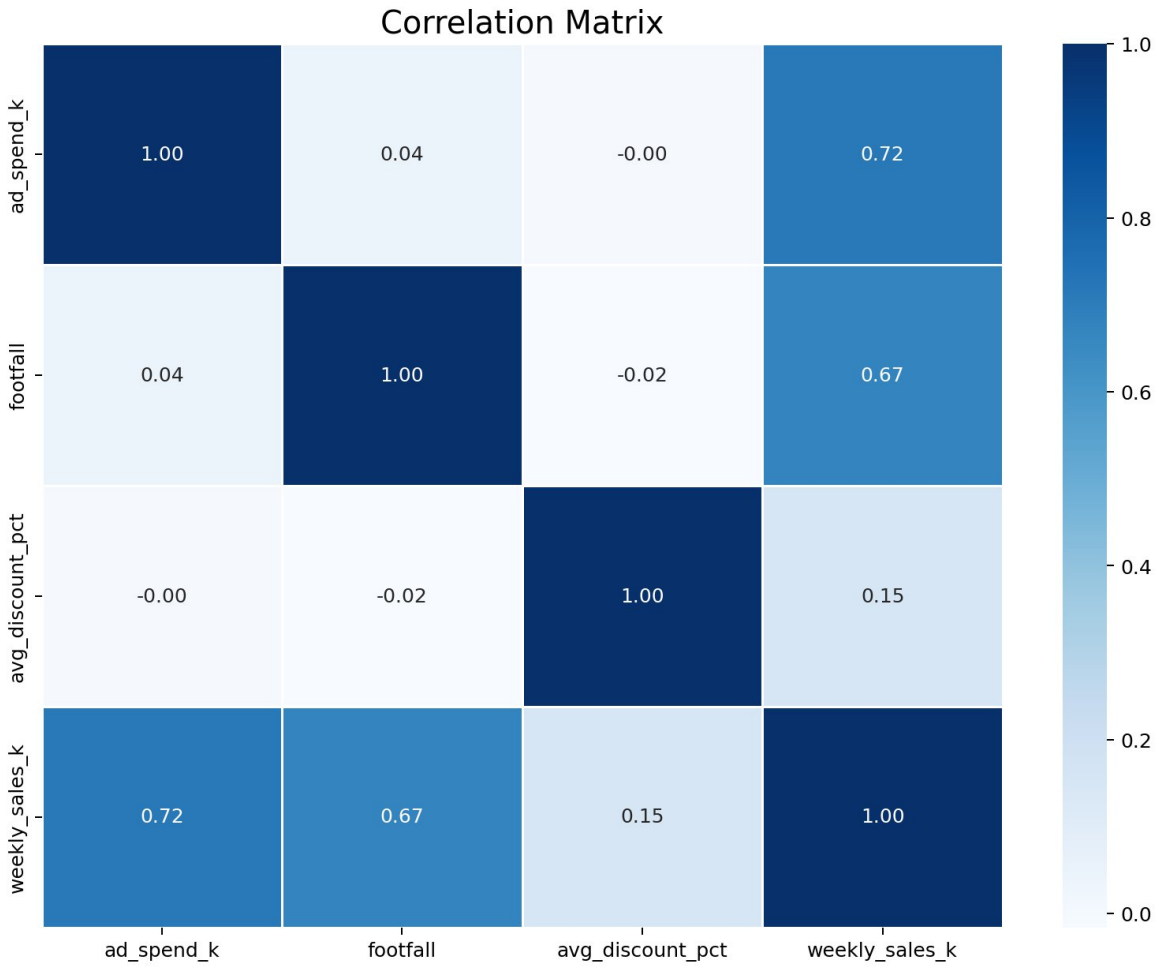
       ad_spend_k  footfall  avg_discount_pct  weekly_sales_k
count       500.0     500.0             500.0           500.0
mean         32.3    2967.6               9.9           449.3
std          16.1    1218.6               5.8           104.6
min           5.6     802.0               0.1           198.9
25%          18.4    1831.8               4.7           372.2
50%          32.7    2968.5              10.1           443.8
75%          46.3    4024.2              14.6           525.9
max          60.0    4997.0              20.0           701.6

Missing values: 0
```

Observation: The dataset loads successfully and contains no missing values.

Step 2: Check the Correlation

```
corr = df.drop(columns="store").corr()
plt.figure(figsize=(6, 4.5))
sns.heatmap(corr, annot=True, cmap="Blues", fmt=".2f")
plt.title("Correlation matrix")
plt.show()
```

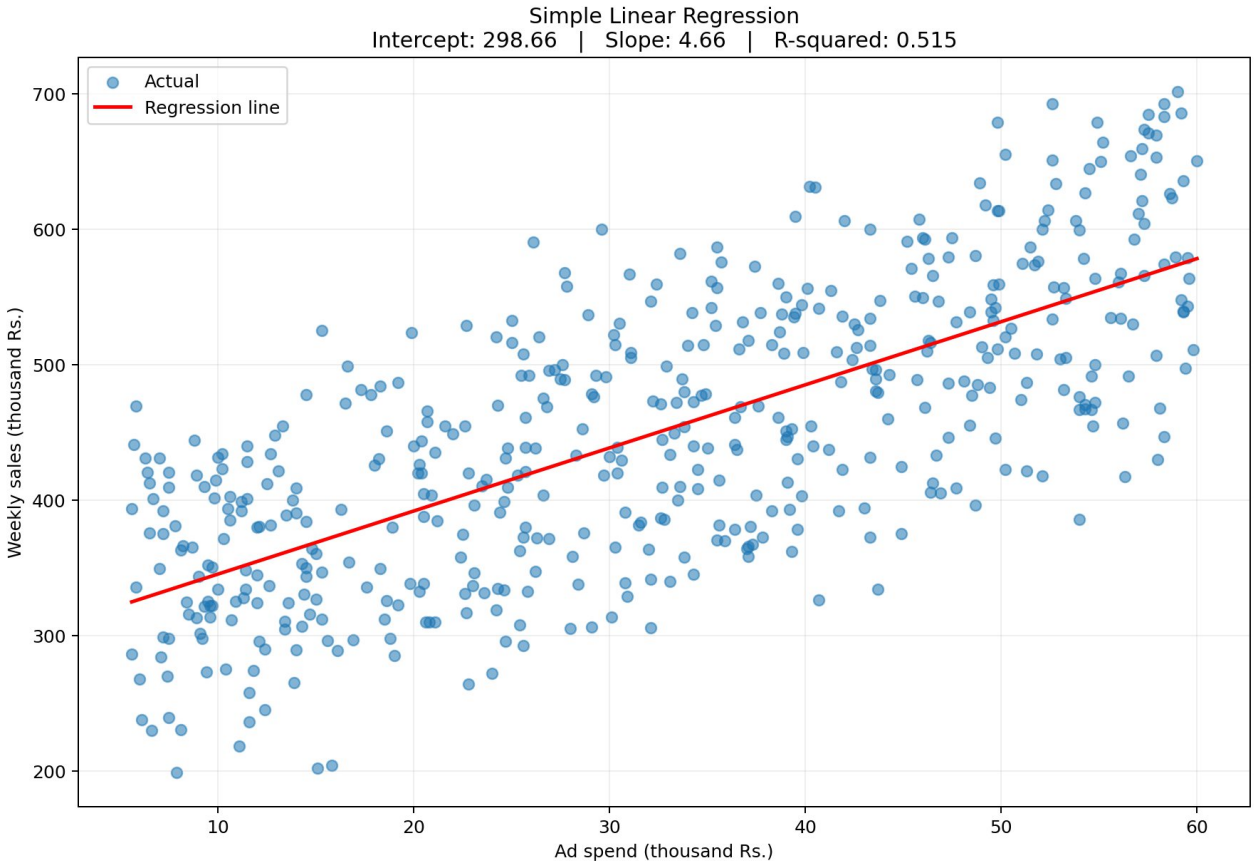


Observation: Weekly sales show positive relationships with advertising spend, footfall and average discount.

Step 3: Simple Linear Regression

```
X1 = df[["ad_spend_k"]]
y = df["weekly_sales_k"]
simple = LinearRegression().fit(X1, y)

print("Intercept:", round(simple.intercept_, 2))
print("Slope (sales per 1k ad spend):", round(simple.coef_[0], 2))
print("R-squared:", round(simple.score(X1, y), 3))
```



Observation: The simple model gives R-squared = 0.515. The fitted line shows the positive relationship between advertising spend and weekly sales.

Step 4: Multiple Linear Regression

```
features = ["ad_spend_k", "footfall", "avg_discount_pct"]
X_train, X_test, y_train, y_test = train_test_split(
    df[features], y, test_size=0.2, random_state=42)
model = LinearRegression().fit(X_train, y_train)
pred = model.predict(X_test)

print("R-squared :", round(r2_score(y_test, pred), 3))
print("MAE       :", round(mean_absolute_error(y_test, pred), 2))
print("RMSE      :", round(np.sqrt(mean_squared_error(y_test, pred)), 2))
print(pd.DataFrame({"Feature": features,
                    "Coefficient": model.coef_.round(3)}))
print("Intercept:", round(model.intercept_, 2))
```

Step 4: Multiple Linear Regression

Python • Google Colab cell

```
features = ["ad_spend_k", "footfall", "avg_discount_pct"]
X_train, X_test, y_train, y_test = train_test_split(
    df[features], y, test_size=0.2, random_state=42)
model = LinearRegression().fit(X_train, y_train)
pred = model.predict(X_test)
```

Output

```
R-squared : 0.964
MAE       : 16.69
RMSE      : 20.47

         Feature  Coefficient
      ad_spend_k        4.504
        footfall        0.055
avg_discount_pct        3.025

Intercept: 109.83
```

Observation: The multiple regression model uses all three predictors and is evaluated on the 20% test set.

Step 5: Statistical Summary using statsmodels

```
Xc = sm.add_constant(df[features])
ols = sm.OLS(y, Xc).fit()
print(ols.summary().tables[1])
print("Adjusted R-squared:", round(ols.rsquared_adj, 3))
```

Step 5: Statistical Summary using statsmodels

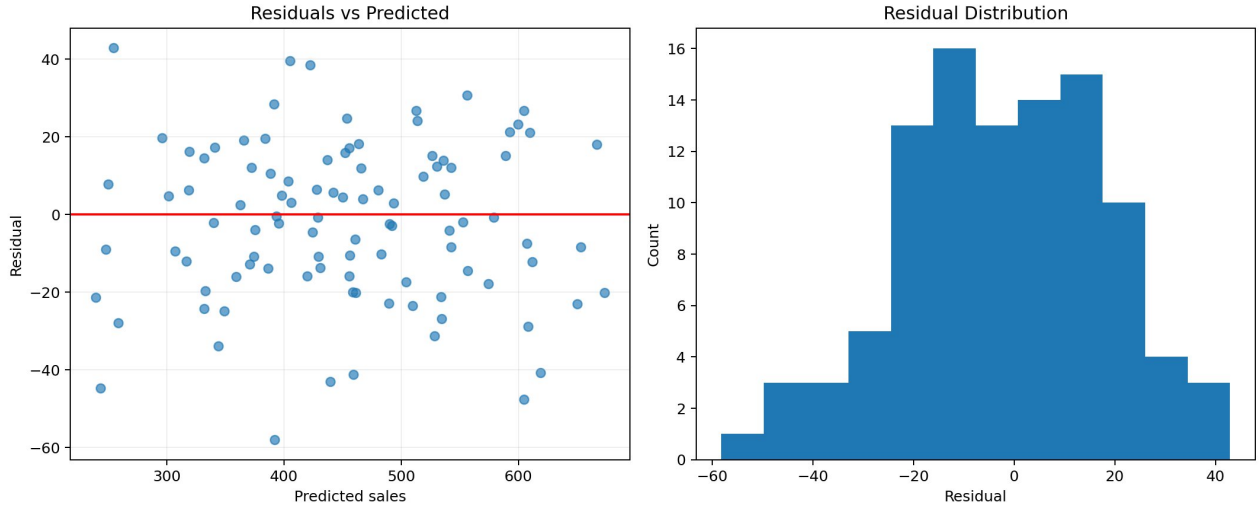
OLS coefficient table

	coef	std err	t	P>|t|	[0.025	0.975]
const	109.7003	3.561	30.808	0.000	102.704	116.696
ad_spend_k	4.4874	0.060	74.832	0.000	4.370	4.605
footfall	0.0556	0.001	70.170	0.000	0.054	0.057
avg_discount_pct	2.9711	0.166	17.933	0.000	2.646	3.297

Adjusted R-squared: 0.958

Step 6: Residual Analysis

```
res = y_test - pred
fig, ax = plt.subplots(1, 2, figsize=(10, 3.8))
ax[0].scatter(pred, res, alpha=0.6)
ax[0].axhline(0, color="red")
ax[0].set_xlabel("Predicted sales")
ax[0].set_ylabel("Residual")
ax[0].set_title("Residuals vs Predicted")
ax[1].hist(res, bins=12)
ax[1].set_title("Residual Distribution")
plt.tight_layout()
plt.show()
```



Observation: The residuals are reasonably scattered around zero, with no strong visible systematic pattern.

Step 7: Predict Weekly Sales for New Data

```
new = pd.DataFrame([[40, 3000, 10]], columns=features)
print("Predicted weekly sales (thousand Rs.):",
      round(model.predict(new)[0], 1))
```

Step 7: Prediction for New Data

Python • Google Colab cell

```
new = pd.DataFrame([[40, 3000, 10]], columns=features)
print("Predicted weekly sales (thousand Rs.):",
      round(model.predict(new)[0], 1))
```

Output

```
Predicted weekly sales (thousand Rs.): 486.3

Equivalent prediction: Rs. 486,305
```

Observation: The model predicts approximately Rs. 486,305 weekly sales for the specified store-week.

Result

Simple and multiple linear regression models were successfully built on the warehouse-style store sales data, evaluated using R-squared, MAE and RMSE, and used to predict weekly sales.

Conclusion

Linear regression quantifies the relationship between sales and business drivers and supports forecasting and planning. The model quality was checked using error metrics and residual analysis.

Practical No. 7

Logistic Regression on Data Warehouse Data (Python)

Aim

To perform logistic regression on customer data from the data warehouse to predict whether a customer will purchase, and evaluate the model.

Software / Tools Used

Python 3, Google Colab, pandas, NumPy, scikit-learn, matplotlib and seaborn

Objectives

- Prepare a dataset with a binary target.
- Build a logistic regression model.
- Evaluate the model using accuracy, confusion matrix and ROC-AUC.
- Interpret coefficients as odds ratios and predict probabilities.

Theory

Logistic regression is a classification technique used when the target has two classes (yes/no). It models the probability of the positive class with the sigmoid function. A record is assigned to the positive class when its predicted probability is at least the threshold, usually 0.5. The exponent of a coefficient, e^b, is the odds ratio: the change in the odds of purchase for a one-unit increase in the variable. The ROC curve plots the true positive rate against the false positive rate at different thresholds. The area under the curve (AUC) summarises performance: 0.5 represents random classification and 1.0 represents perfect classification.

Dataset

customers_ml_logistic_regression.csv (generated, 1,000 rows). The dataset contains customer attributes and a binary purchased target.

Column	Type	Description
age	Numeric	Customer age
annual_income_k	Numeric	Annual income in thousand Rs.
visits_per_month	Numeric	Monthly website/store visits
discount_used	Binary	1 if discount was used, otherwise 0
tenure_months	Numeric	Customer tenure in months
purchased	Binary	Target: 1 = purchased, 0 = did not purchase

Procedure

Step 1: Load the Data

Load the customer dataset and inspect the first rows and class distribution.

```
import pandas as pd, numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.metrics import ConfusionMatrixDisplay, roc_curve, roc_auc_score

pd.set_option("display.width", 110)
pd.set_option("display.max_columns", 20)

df = pd.read_csv("customers_ml_logistic_regression.csv")
print(df.head())
print(df["purchased"].value_counts())
print("Purchase rate:", round(df["purchased"].mean(), 3))
```

Expected/observed output: 1,000 customers are loaded. Purchase rate = 0.289.

Customer data - output

```
   age  annual_income_k  visits_per_month  discount_used  tenure_months  purchased
0   58            618.2                 7              1              5          1
1   48            511.7                 4              1             34          1
2   34            562.0                 5              1             22          0
3   27            351.2                 6              0             12          0
4   40            886.9                 7              0             31          0

purchased
0    711
1    289
Name: count, dtype: int64

Purchase rate: 0.289
```

Step 2: Split the Data

Select the features and target and create training and testing sets.

```
features = ["age", "annual_income_k", "visits_per_month",
            "discount_used", "tenure_months"]
X, y = df[features], df["purchased"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Training shape:", X_train.shape)
print("Testing shape :", X_test.shape)
```

Expected/observed output: 800 training rows and 200 testing rows.

Split - output

```
Training shape: (800, 5)
Testing shape : (200, 5)

Training rows = 800
Testing rows  = 200
```

Step 3: Train the Logistic Regression Model

Fit the logistic regression model and generate class predictions and probabilities.

```
model = LogisticRegression(max_iter=2000)
model.fit(X_train, y_train)

pred = model.predict(X_test)
prob = model.predict_proba(X_test)[:, 1]

print("Accuracy:", round(accuracy_score(y_test, pred), 3))
print("ROC-AUC :", round(roc_auc_score(y_test, prob), 3))
```

Note: predict() gives the class (0/1) using a 0.5 threshold, while predict_proba() gives the probability of class 1.

Model performance - output

```
Accuracy: 0.765
ROC-AUC:  0.815
```

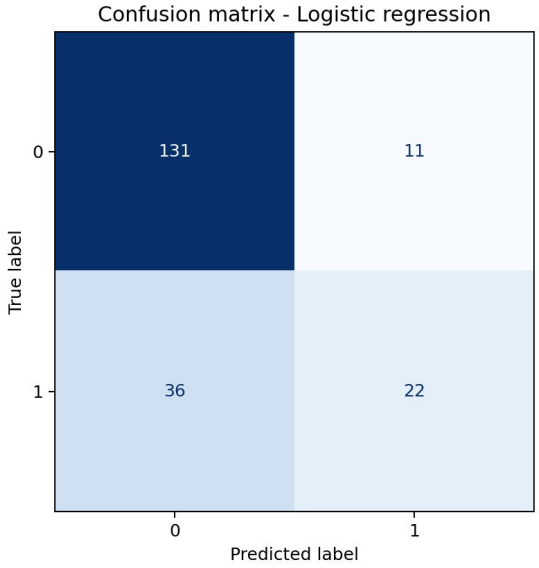
Step 4: Confusion Matrix and Classification Report

Evaluate the model in detail using the confusion matrix and classification report.

```
print(confusion_matrix(y_test, pred))
print(classification_report(y_test, pred))

ConfusionMatrixDisplay.from_predictions(
    y_test, pred, cmap="Blues"
)
plt.title("Confusion matrix - Logistic regression")
plt.show()
```

Observation: The confusion matrix shows true negatives, false positives, false negatives and true positives. Precision, recall and F1-score provide class-wise evaluation.



Classification report

	precision	recall	f1-score	support
0	0.78	0.92	0.85	142
1	0.67	0.38	0.48	58
accuracy			0.77	200
macro avg	0.73	0.65	0.67	200
weighted avg	0.75	0.77	0.74	200

Step 5: ROC Curve and ROC-AUC

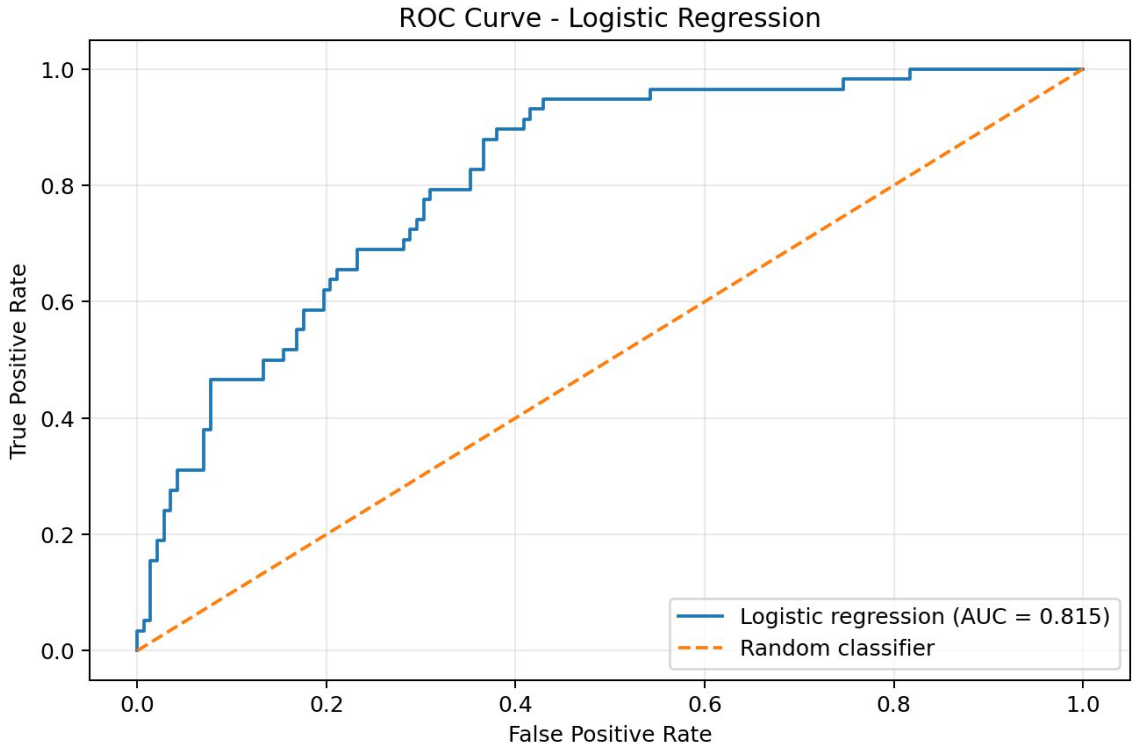
Plot the ROC curve and calculate the area under the curve.

```
fpr, tpr, thresholds = roc_curve(y_test, prob)
auc = roc_auc_score(y_test, prob)

plt.figure(figsize=(7, 5))
plt.plot(fpr, tpr, label=f"Logistic regression (AUC = {auc:.3f})")
plt.plot([0, 1], [0, 1], linestyle="--", label="Random classifier")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Logistic Regression")
plt.legend()
plt.show()

print("ROC-AUC:", round(auc, 3))
```

Observation: The ROC curve gives an AUC of 0.815, indicating useful discrimination between purchasers and non-purchasers.



Step 6: Interpret Coefficients as Odds Ratios

Exponentiate the model coefficients to interpret the effect of each feature as an odds ratio.

```
odds_ratios = np.exp(model.coef_[0])

coef_table = pd.DataFrame({
    "Feature": features,
    "Coefficient": model.coef_[0],
    "Odds Ratio": odds_ratios
})

print(coef_table.round(3))
```

Coefficients and odds ratios - output

```
          Feature Coefficient Odds Ratio
              age       0.058      1.060
 annual_income_k       0.003      1.003
visits_per_month       0.292      1.339
   discount_used       0.773      2.165
   tenure_months       0.014      1.014
```

Step 7: Predict for a New Customer

Predict the probability of purchase for a customer aged 35, income 900k, 6 visits per month, who used a discount, with 24 months tenure.

```
new = pd.DataFrame(
    [[35, 900, 6, 1, 24]],
    columns=features
)

p = model.predict_proba(new)[0, 1]

print(
    "Probability of purchase:",
    round(p, 3),
    "->",
    "Will purchase" if p >= 0.5 else "Will not purchase"
)
```

Expected/observed output: Probability of purchase = 0.619 → Will purchase.

Prediction - output

```
New customer:
age = 35, annual_income_k = 900,
visits_per_month = 6, discount_used = 1,
tenure_months = 24

Probability of purchase: 0.619
Prediction: Will purchase
```

Result

A logistic regression model was built on customer data to predict purchase. The model achieved an accuracy of 0.765 and ROC-AUC of 0.815. The model was evaluated using the confusion matrix, classification report and ROC curve, and its coefficients were interpreted using odds ratios.

Conclusion

Logistic regression predicts the probability of a binary outcome and provides interpretable odds ratios. In this practical, the model was used to identify customers likely to purchase, which can support customer targeting and business decision-making.

Practical No. 8

Reading a CSV File and Data Analysis using Python Pandas

Aim

To write a Python program that reads data from a CSV file, performs data analysis using Pandas, checks the structure and quality of the data, summarizes the records, and generates a simple visualization.

Software / Tools Used

• Python 3
• Google Colab / Jupyter Notebook
• Pandas
• Matplotlib

Objectives

- Read a CSV file into a Pandas DataFrame.
- Display and inspect the first few records.
- Check the shape and column names of the dataset.
- Identify missing values.
- Count records belonging to each species.
- Visualize the number of flowers in each species using a bar chart.

Theory

Pandas is a Python library used for data manipulation and analysis. Its main data structure, the DataFrame, stores data in labelled rows and columns. A CSV file can be read using the read_csv() function. Functions such as head(), shape, columns, isnull().sum(), and value_counts() are used to inspect and summarize data. Matplotlib can be used to represent the analyzed data graphically.

Dataset

The Iris flower dataset is used for this practical. It contains 150 observations belonging to three species: setosa, versicolor and virginica. Each observation contains four numerical measurements: sepal length, sepal width, petal length and petal width, along with the species label.

Procedure

Step 1: Import Libraries

```
import pandas as pd
import matplotlib.pyplot as plt
```

Step 2: Load the CSV File

```
df = pd.read_csv("iris.csv")
print(df.head())
```

Output:

Output

```
   sepal_length  sepal_width  petal_length  petal_width species
0           5.1          3.5           1.4          0.2  setosa
1           4.9          3.0           1.4          0.2  setosa
2           4.7          3.2           1.3          0.2  setosa
3           4.6          3.1           1.5          0.2  setosa
4           5.0          3.6           1.4          0.2  setosa
```

Step 3: Check Dataset Shape

```
print(df.shape)
```

Output:

Output

```
Dataset shape:
(150, 5)
```

Step 4: Display Column Names

```
print(df.columns)
```

Output:

Output

```
Index(['sepal_length', 'sepal_width', 'petal_length', 'petal_width', 'species'], dtype='object')
```

Step 5: Check Missing Values

```
print(df.isnull().sum())
```

Output:

Output

```
sepal_length    0
sepal_width     0
petal_length    0
petal_width     0
species         0
```

Step 6: Count Records by Species

```
print(df["species"].value_counts())
```

Output:

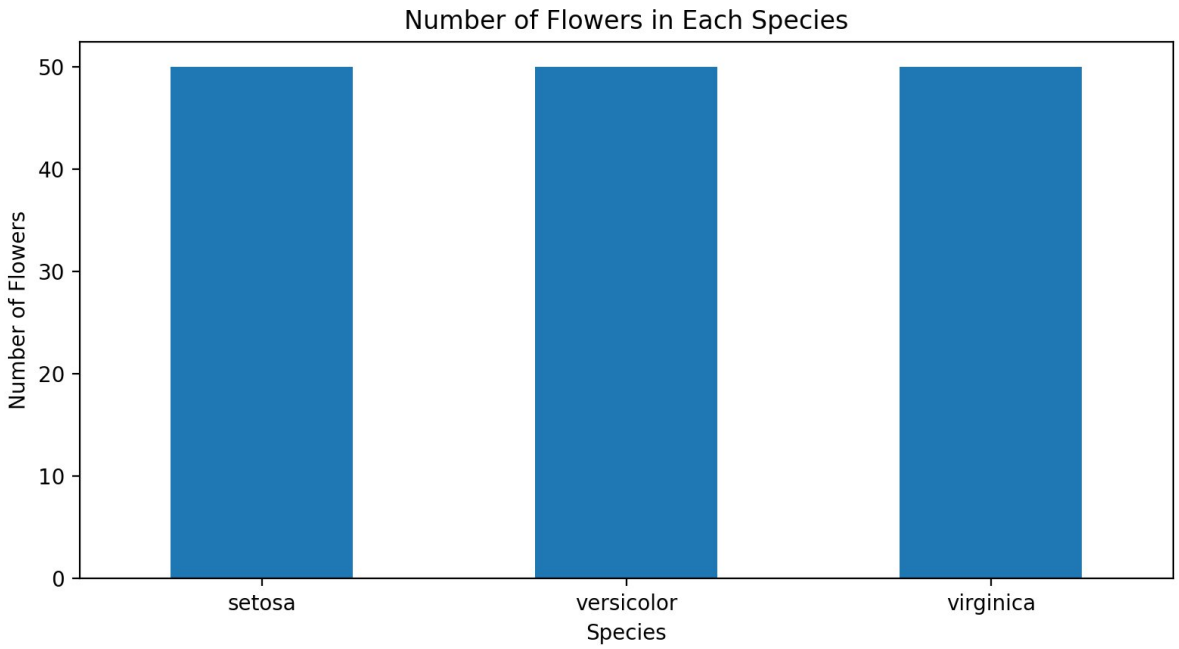
Output

```
species
setosa        50
versicolor    50
virginica     50
```

Step 7: Visualize Species Distribution

```
df["species"].value_counts().plot(kind="bar")
plt.title("Number of Flowers in Each Species")
plt.xlabel("Species")
plt.ylabel("Number of Flowers")
plt.show()
```

Output:



Result

The Iris CSV dataset was successfully loaded and analyzed using Pandas. The dataset contains 150 records and 5 columns. No missing values were found, and each Iris species contains 50 records. A bar chart was generated to visualize the species distribution.

Conclusion

Pandas provides simple and efficient functions for reading, inspecting, validating and summarizing CSV data. In this practical, the Iris dataset was successfully analyzed and visualized using Python.